

Proyectos Semestre 2 · 2026

Descripción de los proyectos de desarrollo

40 proyectos · 160 alumnos

Documento generado a partir de la planilla "Proyectos S2 2026" y de la documentación disponible en el directorio "Proyectos GPI". Agosto de 2026.

Introducción

Este documento describe los proyectos que se desarrollarán durante el segundo semestre de 2026. Cada ficha resume el objetivo de negocio del proyecto, su funcionamiento, los componentes que lo integran y la tecnología que utiliza, sobre la base de la documentación real disponible en la carpeta de cada proyecto (planes de negocio, especificaciones funcionales y de API, planes de desarrollo y manuales). Cuando un proyecto no cuenta con carpeta o esta se encuentra vacía, la ficha se elaboró a partir de la descripción de la planilla y del contexto de la plataforma sobre la que se construye, y así se indica expresamente en el texto.

La planilla contempla 40 proyectos con un total de 160 alumnos. Los proyectos de modalidad "Continuidad" parten de un sistema ya construido en semestres anteriores, mientras que los de modalidad "Nuevo" comienzan su primera versión.

Resumen de proyectos

Proyecto	Alumnos	Modalidad	Carpeta en el directorio
Alloxentric Real Time Agent	4	Nuevo	Carpeta vacía
Aprueba App	3	Nuevo	Aprueba
Aprueba Administracion	3	Nuevo	Aprueba
Barb	4	Continuidad	Barb
Chatbot de Soporte	3	Nuevo	Carpeta vacía
Clear BidNJ	4	Nuevo	ClearBiDNJ
Corestream	3	Continuidad	corestream
Corestream App	3	Nuevo	corestream
Dani 42001	3	Nuevo	Dani 42001,
Dani Mejoras de arquitectura	4	Nuevo	Soporte arquitectura multnorma Dani
Dani GDPR	3	Nuevo	Definicon de frameworks
Dani App Scanner	4	Nuevo	Dani App Scanner
Dani ETH	4	Continuidad	Dani ETH
Directa v2	3	Continuidad	Directa Asistente IA para directores de empresa
Dossier	3	Continuidad	Dossier Project
Galactum equipo 1	3	Nuevo	Galactum
Galactum equipo 2	3	Nuevo	Galactum
Galactum equipo 3	3	Nuevo	Galactum
Galactum equipo 4	3	Nuevo	Galactum
Galactum equipo 5	3	Nuevo	Galactum
Hytro-Sports	3	Nuevo	Hytro - sports
Inteligencia de Mercado	3	Continuidad	Inteligencia de mercado Encipharm
KalliCode equipo 1	4	Nuevo	KalliCode
KalliCode equipo 2	4	Nuevo	KalliCode
KalliCode equipo 3	4	Nuevo	KalliCode
KalliCode equipo 4	4	Nuevo	KalliCode
KalliCode equipo 5	4	Nuevo	KalliCode

Proyecto	Alumnos	Modalidad	Carpeta en el directorio
KalliCode equipo 6	4	Nuevo	KalliCode
ND Sales Assistant v2	4	Continuidad	Sales Assitant NoonDalton
SSO Gestion de riesgo Industrial v2	4	Continuidad	SSO sistema de gestion de riesgos de planta industrial
Vox Ready	3	Nuevo	VoxReady
Vox Ready Administracion	3	Nuevo	Vox Ready
Acredittia	3	Nuevo	Acredittia
Acredittia administracon del negocio	3	Nuevo	Acredittia
Agecare	3	Nuevo	AgeCare
Agecare modelo de generacion de musica	3	Nuevo	AgeCare
Agecare (administración)	3	Nuevo	AgeCare
TryAI Gap	4	Nuevo	TryAIGap
WellQ App Exámenes médicos	4	Continuidad	Carpeta vacía (WellQ Upload Cinical Test)
WellQ App GLP1 Skin	4	Continuidad	Carpeta vacía (WellQ GLP-1)
WellQ API	4	Continuidad	Proyecto APIs WellQ
WellQ Generic App for manufacturers	4	Nuevo	Sin carpeta
WellQ AI driven Exercise Generator	4	Nuevo	Sin carpeta
Paralegal	4	Nuevo	Aplicacion de Asistencia Legal para empresas

Proyectos a los que les falta carpeta

Aun en fase de completar la información de los siguientes proyectos

- **Dani SOC2**
- **Dani GDPR**
- **WellQ Generic App for manufacturers**
- **WellQ AI driven Exercise Generator**
- Alloxentric Real Time Agent
- Chatbot de Soporte (carpeta "Chatbot de soporte generico Alloxentric")
- WellQ App Exámenes médicos (carpeta "WellQ Upload Clinical Test")
- WellQ App GLP1 Skin (carpeta "WellQ GLP-1")

Voy a crear y poblar estas carpetas con, al menos, una descripción del problema de negocio, la especificación funcional y el plan de desarrollo antes del inicio del semestre, de modo que los equipos partan con la misma base documental que el resto de los proyectos.

Índice de proyectos

Introducción.....	2
Resumen de proyectos	2
Proyectos a los que les falta carpeta	4
Índice de proyectos	5
Alloxentric Real Time Agent.....	7
Aprueba Administracion.....	8
Barb.....	9
Chatbot de Soporte	10
Clear BidNJ.....	11
Corestream	12
Dani 42001.....	13
Dani SOC2	14
Dani GDPR.....	15
Dani App Scanner	16
Dani ETH	17
Directa v2.....	18
Dossier	19
Galactum equipo 1	20
Galactum equipo 2	21
Galactum equipo 3	22
Galactum equipo 4	23
Galactum equipo 5	24
Galactum Equipo 6.....	25
Hydro-Sports	26
Inteligencia de Mercado.....	27
ND Sales Assistant v2.....	33
SSO Gestion de riesgo Industrial v2.....	34
Vox Ready	35
Acredittia	36
Acredittia administracon del negocio.....	37
Agecare	38
Agecare modelo de generacion de musica.....	39
Agecare (administración).....	40
TryAI Gap	41
WellQ App Exámenes medicos	42
WellQ App GLP1 Skin	43
WellQ API.....	44
WellQ Generic App for manufacturers	45
WellQ AI driven Exercise Generator	46

Chat de Soporte Generico Alloxicentric	47
1. Introducción.....	47
2. Objetivos.....	47
3. Alcance.....	48
4. Cómo funciona.....	49
5. Arquitectura de la solución.....	49
Proyecto Paralegal (Justiniano) 1. ¿Qué es Justiniano?	51
2. Objetivo de negocio.....	51
3. ¿Cómo funciona?	51
4. Componentes del sistema	52
5. Tecnología utilizada	53

Alloxentric Real Time Agent

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: carpeta vacía

Objetivo de negocio

Alloxentric es una empresa especializada en automatización de comunicaciones por voz (voicebots) para canales como telefonía y atención al cliente. Según la descripción de la planilla, el proyecto consiste en el desarrollo de un engine de agente conversacional usando ElevenLabs y Tacotron. No existe documentación adicional en la carpeta del proyecto, por lo que esta ficha describe el alcance propuesto a partir de esa descripción y del dominio de la empresa.

Se propone construir el motor (engine) que permita sostener conversaciones de voz en tiempo real con clientes: un agente capaz de escuchar, comprender y responder hablando con una voz natural y con latencia suficientemente baja para que el diálogo se sienta fluido, como una llamada con una persona. Para Alloxentric, un engine propio de este tipo es una pieza estratégica: habilita casos de uso de cobranza, agendamiento, encuestas y soporte de primer nivel, reduciendo costos frente a call centers tradicionales y permitiendo comparar y combinar proveedores de síntesis de voz (ElevenLabs como servicio comercial de alta calidad y Tacotron como modelo neuronal abierto auto-hospedado) según costo, latencia y calidad.

Cómo funciona

El flujo propuesto sigue el pipeline clásico de un agente de voz en tiempo real: el audio del usuario llega en streaming (por ejemplo desde telefonía SIP/WebRTC o un micrófono web); un módulo de reconocimiento de voz (ASR/STT) lo transcribe de forma incremental, con detección de actividad de voz y de fin de turno; un LLM orquestado con el contexto del negocio (prompt, historial, herramientas) genera la respuesta del diálogo; y la síntesis de voz (TTS) convierte esa respuesta en audio, contemplándose ElevenLabs vía API de streaming y Tacotron (con un vocoder neuronal) como alternativa auto-hospedada. El engine se propone como orquestador de todo el ciclo escuchar→pensar→hablar, gestionando la conversación por turnos, las interrupciones del usuario (barge-in) y la reproducción del audio a medida que se genera, con el objetivo de mantener la latencia de respuesta percibida en el orden de uno a dos segundos.

Componentes

Como alcance propuesto: (1) gateway de audio bidireccional en streaming (WebSocket/WebRTC o integración telefónica); (2) módulo ASR/STT en streaming con VAD y detección de turno; (3) orquestador de diálogo basado en LLM con gestión de contexto y estados; (4) capa de TTS con doble proveedor —ElevenLabs y Tacotron— intercambiable tras una interfaz común; (5) métricas de latencia y calidad por componente; y (6) una aplicación de demostración (por ejemplo, un caso de atención al cliente) para validar el engine de punta a punta.

Tecnología

Se propone un backend en Python (FastAPI o similar) con WebSockets/WebRTC para el streaming de audio; ASR en streaming (Whisper u otro servicio cloud); un LLM vía API para el diálogo; ElevenLabs mediante su API de streaming y Tacotron 2 con vocoder (p. ej. HiFi-GAN o WaveGlow) servido con PyTorch/GPU para la síntesis auto-hospedada. El foco de ingeniería está en la arquitectura de streaming de baja latencia, el manejo de concurrencia y la evaluación comparativa de ambos motores de voz. El alcance definitivo deberá acordarse con la empresa al inicio del proyecto.

Aprueba Administracion

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Aprueba

Objetivo de negocio

Aprueba es una plataforma freemium (móvil y web) de práctica intensiva para estudiantes que rinden exámenes estandarizados de admisión universitaria, con mercado inicial en Chile (PAES, con más de 308.000 inscritos en 2025) y expansión proyectada al Reino Unido (UCAT, LNAT, A-Levels). Su modelo entrega un flujo diario de preguntas desde un banco central, con desbloqueo progresivo a cambio de datos del estudiante (10 preguntas con nombre y correo, hasta 20 al completar el perfil) y suscripciones de bajo costo: US\$1/mes por una prueba y US\$5/mes por todas.

Este proyecto no construye la app del alumno, sino los **módulos de Administración**: la consola web con la que el equipo de negocio y operaciones monitorea y gestiona el producto. Sin ella, la empresa no puede medir conversión ni ingresos, gestionar sponsors, moderar contenido ni operar la plataforma; el plan de negocio destaca además que el administrador debe poder ajustar la economía del producto (límites diarios, precios, catálogo de pruebas) sin desarrollo adicional.

Cómo funciona

Un usuario administrativo inicia sesión con correo, contraseña y segundo factor (OTP/TOTP), y la consola le muestra solo las secciones que su rol permite (gerencia, finanzas, operaciones, soporte). Desde el **Resumen** ve KPIs (descargas, MAU, convertidos, MRR) y la salud del sistema; en **Comercial** analiza el embudo de conversión, la recaudación por plan y exporta métricas a CSV (acceso restringido a finanzas); en **Sponsors** gestiona auspiciadores y beneficios; en **Operativa** monitorea tiendas, servicios y contenedores, con acción de reinicio auditada; en **Usuarios y soporte** busca usuarios, aplica acciones (suspender, cambiar plan), gestiona tickets y modera solicitudes de corrección (confirmar otorga 250 medallas de bronce al alumno); y en **Contenido/Planes** administra el banco de preguntas con importación masiva JSON y un constructor de planes con arrastrar y soltar, límites configurables y sincronización de precios con Stripe. Toda acción sensible queda registrada en una bitácora de auditoría.

Componentes

SPA de administración (8 vistas: Resumen, Comercial, Sponsors, Operativa, Usuarios y soporte, Tutores, Contenido, Planes y productos); backend administrativo que expone ~33 endpoints bajo `/admin` con agregación de métricas, RBAC, auditoría y control de contenedores; y dependencia de los datos emitidos por los servicios del alumno y de pagos.

Tecnología

Frontend en React 18 + Vite + React Router (JavaScript), con tema claro/oscuro, i18n es/en y gráficos ligeros (SVG/librería liviana). Backend en Node.js + Express con Firebase Admin SDK sobre Firestore (con opción de emulador local), JWT con claims de rol y MFA, envelope común `{data, error, meta}` bajo `/api/v1`, e integración con Stripe para precios. Plan Scrum de 10 sprints semanales (1 FE + 1 BE, ~8 SP por desarrollador/sprint) según el Tomo III del plan de desarrollo.

Barb

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Barb

Objetivo de negocio

Barb es una plataforma SaaS de mantenimiento industrial inteligente. Ataca uno de los mayores sumideros de valor de la manufactura: las paradas no planificadas, cuyo costo promedio oscila entre US\$1.000 y US\$5.000 por minuto (una detención de cuatro horas puede superar los US\$480.000). Las causas raíz son sobre todo informacionales: conocimiento fragmentado en manuales dispersos, dependencia de técnicos expertos, ausencia de trazabilidad de intervenciones y hasta un 30% del tiempo de los técnicos consumido en burocracia. Barb transforma la gestión de mantenimiento de reactiva a preventiva y predictiva, poniendo documentación contextual, diagnóstico asistido y memoria institucional al alcance de operadores, técnicos e ingenieros de planta. Su modelo es multi-tenant con partición por cliente y suscripción por niveles (US\$3.000–8.000/mes según planta), con ROI proyectado desde el primer evento crítico resuelto. Al ser un proyecto de continuidad, el equipo desarrolla mejoras y nuevas funcionalidades sobre una base existente (se entrega un Resource Pack funcional que debe elevarse a grado de producción).

Cómo funciona

Tras autenticarse, el operador accede a un dashboard con tres vías: **Document Chat** (consulta conversacional de manuales y procedimientos, filtrando en cascada por planta, disciplina técnica —eléctrica, mecánica, hidráulica, neumática, automatización— y máquina), **Machine Debug** (sesión interactiva de diagnóstico de un equipo: el operador describe síntomas, adjunta fotos y el sistema sugiere causas combinando documentación e historial, generando al cierre un reporte estructurado) y **Topología de Planta** (grafo interactivo de dependencias entre equipos, útil para fallas en cascada). El módulo de órdenes de trabajo gestiona el ciclo completo en seis estados (Creada → Cerrada) con registro de tiempos, repuestos y evidencia fotográfica: los técnicos ejecutan desde la app móvil y los supervisores monitorean desde la web. Dashboards de KPIs calculan MTBF, MTTR y costos por equipo. Un RBAC de cuatro roles (Administrador, Gerente, Técnico, Visualizador) delimita el acceso. Cada intervención alimenta la base de conocimiento, habilitando con el tiempo el mantenimiento predictivo.

Componentes

Aplicación web de supervisión y configuración; aplicación móvil con soporte offline para técnicos en terreno; backend API REST centralizado con motor de reglas; módulos de documentación/chat, diagnóstico, topología, órdenes de trabajo, inventario de repuestos y analítica; integración con los servicios de comunicación e IA del ecosistema Alloxxentric.

Tecnología

Según la documentación: web en Vue 3 (Composition API) + TypeScript estricto, Pinia, Tailwind CSS/Headless UI, Chart.js y Axios con interceptores; app móvil en Flutter/Dart; backend en FastAPI (Python) con PostgreSQL y Redis, JWT y consumo de modelos de NLP/análisis predictivo. El plan de desarrollo asociado (8 sprints, equipo de 4: 2 backend + 2 frontend) contempla además una variante Node.js/Express + React 18 con Docker, GitHub Actions, AWS (ECS, RDS, S3) y Terraform. Entregables de continuidad: refinamiento de UI, manuales de usuario y de despliegue (Docker), depuración y QA.

Chatbot de Soporte

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: carpeta vacía

Objetivo de negocio

Según la planilla oficial, el proyecto consiste en el "desarrollo de un bot conversacional de soporte conectado a una ticketera genérica (implementada para Corestream) para varios SaaS: WellQ, Alloxentric, Dani ETH, Dani27001 y Aprueba". El problema de negocio es claro: cada producto SaaS del ecosistema necesita atender consultas y problemas de sus usuarios, y hacerlo con soporte humano dedicado por producto es costoso y no escala. La propuesta es una primera línea de atención automatizada y compartida: un bot conversacional capaz de responder preguntas frecuentes y, cuando no puede resolver, registrar el caso como ticket en una ticketera común, de modo que un mismo mecanismo de soporte sirva a múltiples productos. Esta dirección es consistente con el ecosistema documentado: la cotización de ClearBid NJ, por ejemplo, ya ofrece un "AI Support Bot" como complemento recomendado para reducir costos operativos, y Corestream —el sistema de control de proyectos y ticketing del grupo— aporta la infraestructura de tickets sobre la que el bot se apoyaría.

Al tratarse de un proyecto nuevo cuya carpeta de documentación está vacía, lo que sigue describe el alcance propuesto y la arquitectura esperada, no un diseño ya especificado.

Cómo funciona

El flujo contemplado es el siguiente: un usuario final de cualquiera de los SaaS abre el canal de soporte (por ejemplo, un widget de chat embebido en la aplicación) y conversa en lenguaje natural con el bot, que respondería a partir de la base de conocimiento del producto correspondiente. Si la consulta excede su capacidad o el usuario lo solicita, el bot crearía un ticket en la ticketera genérica —con la conversación como contexto, el producto de origen y una clasificación inicial de prioridad— y lo derivaría al equipo humano. Los agentes gestionarían los tickets desde la ticketera (estados, asignación, respuesta), y el usuario recibiría seguimiento de su caso. Se propone que la solución sea multi-tenant: una sola implementación configurable por producto (marca, idioma, base de conocimiento y reglas de derivación por cada SaaS atendido).

Componentes

Se contempla: el motor conversacional del bot; conectores/widget de chat para integrarse en cada SaaS; la integración con la ticketera genérica (creación, consulta y actualización de tickets vía API); un módulo de administración para configurar tenants, bases de conocimiento y reglas de escalamiento; y paneles básicos de métricas de atención.

Tecnología

La planilla no fija el stack, por lo que se propone alinearse con el ecosistema existente: backend en Python con FastAPI, PostgreSQL y Redis (el stack de Corestream, cuya ticketera expone API REST, JWT/RBAC y notificaciones en tiempo real por WebSocket), frontend/widget en un framework SPA moderno (Vue 3 o React con TypeScript), y un componente de IA conversacional basado en modelos de lenguaje (LLM) con recuperación sobre la documentación de cada producto (RAG). Las decisiones definitivas de arquitectura e integración forman parte del trabajo del equipo.

Clear BidNJ

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: ClearBiDNJ

Objetivo de negocio

ClearBid NJ es un sistema de información de propiedades en remate del Estado de New Jersey (EE. UU.), desarrollado para el cliente ClearBid NJ, LLC. Los inversionistas inmobiliarios que participan en subastas (sheriff sales, tax liens, tax deeds, propiedades de bancos) deben evaluar antes de pujar información financiera, legal y física dispersa y difícil de verificar: gravámenes que sobreviven a la subasta, estado del título, ocupación, valor tras reparación (ARV) y retorno esperado. El sistema convierte cada propiedad en una ficha verificada de "inteligencia pre-puja": permite explorar oportunidades por county o código postal, revisar el detalle completo y apoyarse en un **semáforo de riesgo** (verde/amarillo/rojo) calculado automáticamente por un motor de reglas. La cotización del proyecto (Alloxentric para ClearBid NJ, US\$16.600 en tres fases: diseño, desarrollo y testing, ~3 meses) confirma que se parte prácticamente desde cero, con superficie de aplicación, sitio público y panel de administración.

Cómo funciona

El inversionista se registra o entra con OAuth (Google, Apple, Facebook) y completa una verificación 2FA por SMS obligatoria. Luego explora las oportunidades en vistas de tabla, tarjetas o mapa, filtrando por county, ZIP, color de semáforo, tipo de remate, tipo de propiedad, rango de precio de apertura y ROI. Cada ficha ofrece cuatro pestañas: financiera (precio de apertura, liens, ARV, reparaciones estimadas, puja máxima sugerida, ROI), legal (tipo y fecha de subasta, número de caso, estado del título, ocupación, depósito, documentos PDF), física (superficie, dormitorios, año, condición, barrio) y semáforo (reglas evaluadas con su severidad). El usuario guarda favoritos, define su área de cobertura (counties/ZIP) y recibe alertas de nuevas propiedades, bajas de precio y recordatorios de subasta (push/email/SMS). La web añade segmentos (filtros guardados con alertas), exportación asíncrona a CSV/Excel, operaciones en lote y facetas. Por el lado interno, analistas y administradores cargan datos mediante trabajos de ingesta, editan propiedades, administran las reglas del motor y fuerzan recálculos del semáforo.

Componentes

Backend API REST (diez módulos: auth/2FA, usuarios, geografía, propiedades, semáforo y reglas, guardadas, cobertura, alertas, administración/ingesta y utilidad); interfaz web de escritorio; aplicación móvil de inversionista contemplada en la especificación; motor de reglas del semáforo con administración propia; y panel/rol administrativo para analistas.

Tecnología

Backend en FastAPI (Python 3.12) con PostgreSQL, SQLAlchemy 2.0 async, Alembic y Pydantic v2; JWT con refresh rotativo, Argon2/bcrypt, OAuth verificado en servidor y 2FA por SMS (Twilio, con stubs conmutables); trabajos asíncronos con BackgroundTasks (Celery/RQ en producción), OpenAPI/Swagger automático, Docker y pytest. Frontend web en React 18 + TypeScript + Vite, Tailwind CSS, TanStack Query, Zustand, Axios con refresh transparente, react-i18next (en/es/zh/de con modo oscuro) y react-leaflet + OpenStreetMap para el mapa. Se sugiere pg_trgm/PostGIS para búsqueda y consultas geográficas, y FCM/APNs para push.

Corestream

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: corestream

Objetivo de negocio

CoreStream es una plataforma de "inteligencia operacional" para la gestión y control de proyectos de desarrollo de software: organiza el trabajo en una jerarquía de Aplicaciones → Épicas → Tickets → Subtareas y va más allá del gestor de tareas tradicional al centrarse en la trazabilidad del actor y la fluidez de la interacción, con mecanismos propios de Preguntas (bloqueos), Redirecciones (traspasos entre desarrolladores) y métricas de rendimiento en tiempo real. La versión 1.0 ya existe —fue construida en 10 sprints semanales por un equipo de 4 desarrolladores— y funciona además como la ticketera de referencia del ecosistema. Al ser un proyecto de continuidad, el objetivo es desarrollar las mejoras y nuevas funcionalidades identificadas en el informe de mejoras potenciales, para llevar el producto de un sistema de seguimiento de tickets funcional a una herramienta de gestión de proyectos competitiva.

Cómo funciona

El sistema distingue tres roles. El **Administrador** trabaja en cuatro pestañas: Project Canvas (constructor visual de aplicaciones, épicas como swimlanes y tickets con drag & drop), Analytics (eficiencia, índice de bloqueo, índice de rotación, mapa de calor de actividad y burndown), Code & Documentation (repositorio de código y documentos con traducción automática a 8 idiomas) y Team Management (altas de desarrolladores y promoción de Líderes de Grupo). El **Desarrollador** opera desde My Workbench: toma sus tickets, marca subtareas, y ejecuta una de tres acciones: Completar (exige enlace de PR válido en GitHub/GitLab/Bitbucket), Preguntar (bloquea el ticket) o Redirigir (lo transfiere con motivo). El **Líder de Grupo** asigna tickets según la carga de cada miembro. Las notificaciones llegan en tiempo real (asignaciones, redirecciones, preguntas, completados). La continuidad prioriza, según el informe de mejoras: soporte de Sprints y planificación ágil (backlog, story points, velocidad), Kanban avanzado (límites WIP, columnas personalizables), dependencias entre tickets, analítica avanzada (burnup, CFD, forecasting), integración bidireccional con Git y CI/CD, integraciones Slack/Teams, MFA/SSO, testing sistemático y PWA/móvil.

Componentes

SPA web con vistas por rol (Constructor, Workbench, Analytics, Team Management, Code & Docs, Team Assignment); API REST asíncrona; base de datos relacional (users, roles, tickets, ticket_events, epics, subtasks, applications, documents, notifications); capa de tiempo real (WebSocket + pub/sub) y cola de tareas; sistema de i18n en 5 idiomas con modo oscuro.

Tecnología

Frontend: Vue 3 (Composition API) + TypeScript + Vite, Pinia, vue-router, Axios, Tailwind CSS, vuedraggable, vue-i18n, Chart.js/ApexCharts y jsPDF; testing con Vitest y Cypress. Backend: Python con FastAPI/Uvicorn, Pydantic v2, SQLAlchemy + asyncpg sobre PostgreSQL, migraciones Alembic, Redis con ARQ (colas y pub/sub de notificaciones), WebSocket nativo, JWT (python-jose) con bcrypt y RBAC (ADMIN, TEAM_LEADER, DEVELOPER); pytest y pytest-asyncio. Despliegue: Vercel (frontend) y Railway (backend, PostgreSQL y Redis) vía Docker, con Git/GitHub para control de versiones y CI/CD.

Dani 42001

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: Dani 42001

Objetivo de negocio

Dani Platform es la plataforma SaaS de cumplimiento normativo (GRC) de Alloxentric: un backend multi-tenant que gestiona políticas, procedimientos, controles, evidencias, análisis de brechas, sala de auditoría y declaración de aplicabilidad (SOA), inicialmente construido alrededor de ISO/IEC 27001. La versión 2 convierte el concepto de "framework" en un eje transversal: cada documento se vincula a cláusulas concretas de una o varias normativas, con una tabla de equivalencias que sostiene la promesa comercial de "un esfuerzo, varias certificaciones". Este proyecto consiste en desarrollar el módulo ISO/IEC 42001, el estándar de sistemas de gestión de inteligencia artificial (AIMS), para que las organizaciones que desarrollan o usan IA puedan gestionar y demostrar su gobernanza de IA desde Dani. La taxonomía de la plataforma ya contempla la categoría ai_governance, por lo que el trabajo es incorporar un framework completo y funcional sobre ese cimiento.

Cómo funciona

El equipo carga el catálogo de cláusulas de ISO/IEC 42001 (cláusulas 4 a 10 y controles del Anexo A) en el modelo de datos de la plataforma: cláusulas como subcolección de Firestore con jerarquía, tipo de cláusula y la marca `requires_documented_information`, que alimenta el informe de "cláusulas obligatorias sin documento aprobado". La documentación del proyecto detalla la información documentada obligatoria del estándar —alcance del AIMS, política de IA, registro de riesgos de IA, metodología de evaluación y tratamiento de riesgos, evaluación de impacto de sistemas de IA, SOA, objetivos de IA, auditoría interna, revisión por la dirección y acciones correctivas— junto con los documentos recomendados (inventario de sistemas de IA, procedimientos de gestión de datos, incidentes y cambios). El módulo debe mapear estos requisitos a los tipos documentales de Dani mediante `framework_links` a nivel de cláusula, calcular la cobertura por cláusula en el agregado materializado (`not_covered`, `documented`, `implemented`, `verified`) y aprovechar la estructura armonizada del Anexo SL para sembrar equivalencias con ISO 27001, de modo que una organización ya certificada reutilice su documentación de gestión.

Componentes

Catálogo de cláusulas y controles del Anexo A de ISO/IEC 42001 con carga masiva idempotente; SOA específica del AIMS; informes de brechas documentales y matriz de cobertura; equivalencias curadas con ISO 27001; integración con los módulos existentes de documentos, controles, evidencias, `gap_analysis`, auditoría y dashboard mediante el filtro uniforme por framework; y coherencia con el módulo vertical euai (Reglamento de IA de la UE) ya presente.

Tecnología

Stack real de Dani Platform: backend FastAPI (Python), organizado en módulos por dominio (`models.py` con Pydantic, `service.py`, `router.py`) bajo `/api/v1`; persistencia en Google Cloud Firestore con agregados recalculados por Cloud Functions; ficheros en Google Cloud Storage con URLs firmadas; autenticación con Keycloak (JWT, roles `super_admin/admin/auditor/contributor`); despliegue en Docker sobre Cloud Run; frontend React con tokens de diseño CSS, tema claro/oscuro e i18n.

Dani SOC2

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: sin carpeta

Objetivo de negocio

Este proyecto no cuenta con carpeta de documentación propia: la ficha describe el alcance propuesto según la planilla oficial —"Desarrollo del módulo SOC2 para Sistema Dani"— sobre la arquitectura documentada de la plataforma. Dani Platform es la solución SaaS de cumplimiento normativo (GRC) de Alloxentric, y su especificación v2 define un marco normativo multi-framework donde SOC 2 figura explícitamente en el catálogo inicial junto a ISO/IEC 27001 e ISO 9001. SOC 2 es el marco de atestación de la AICPA más solicitado por clientes corporativos de empresas SaaS en Norteamérica, de modo que ofrecerlo dentro de Dani amplía directamente el mercado de la plataforma. El proyecto se propone implementar SOC 2 como framework de primera clase: catálogo de criterios de servicios de confianza (Trust Services Criteria) con sus puntos de foco, mapeo de documentos y controles, y presentación correcta en el Trust Center.

Cómo funciona

Sobre el modelo de datos ya especificado, el equipo cargaría el catálogo SOC 2 como subcolección de cláusulas con `clause_type = point_of_focus` para los puntos de foco y la marca certifiable = false, distinción que la especificación exige para que el Trust Center no rotule como "certificación" lo que es una atestación. Se contempla poblar la tabla de equivalencias `ClauseEquivalence` entre criterios SOC 2 y controles de ISO 27001 (la propia especificación usa como ejemplo que A.5.15 y CC6.1 piden en esencia lo mismo), de forma que un cliente ya certificado en ISO 27001 vea cuantificada la reutilización de sus políticas y evidencias mediante la matriz de cobertura y el informe de reutilización (GET /coverage/reuse-report). El módulo debe integrarse con el filtro por framework de los módulos existentes (documentos, controles, evidencias, análisis de brechas, sala de auditoría y dashboard) y con el flujo de publicación curada del Trust Center, donde el informe SOC 2 es un documento típicamente restringido bajo NDA con token de un solo uso y marca de agua.

Componentes

Se propone: catálogo de Trust Services Criteria con carga masiva; `FrameworkScope` por organización (estado `evaluating/in_scope`, periodo de atestación); equivalencias curadas con ISO 27001; adaptación de la sala de auditoría al árbol de criterios SOC 2; e integración con las tarjetas del Trust Center distinguiendo atestación de certificación.

Tecnología

Stack real de Dani Platform según la documentación: FastAPI (Python) con módulos por dominio (Pydantic, service, router) bajo /api/v1; Google Cloud Firestore con agregados materializados vía Cloud Functions; Google Cloud Storage con URLs firmadas; Keycloak para autenticación JWT y roles; Docker y Cloud Run en Google Cloud; frontend React con tokens de diseño, tema claro/oscuro e internacionalización.

Dani GDPR

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: sin carpeta

Objetivo de negocio

Este proyecto no dispone de carpeta de documentación específica: la ficha describe el alcance propuesto a partir de la planilla oficial —"Desarrollo del módulo GDPR para Sistema Dani"— sobre la plataforma existente y documentada. Dani Platform, la solución SaaS de cumplimiento normativo (GRC) de Alloxxentric, gestiona hoy frameworks certificables (ISO 27001, ISO 9001) y de atestación (SOC 2); su especificación v2 reconoce expresamente al GDPR como una tercera familia: las regulaciones de cumplimiento obligado (certificable = false, familia eu_regulation, categoría privacy). El Reglamento General de Protección de Datos es de aplicación obligatoria para cualquier organización que trate datos personales de residentes europeos, y su incumplimiento acarrea sanciones significativas, por lo que un módulo GDPR convierte a Dani en herramienta de trabajo cotidiana también para los responsables de privacidad, no solo para los de seguridad.

Cómo funciona

Se propone incorporar el GDPR al catálogo multi-framework de la plataforma: sus artículos y obligaciones se cargarían como subcolección de cláusulas con jerarquía, marcando con `requires_documented_information` las que exigen documentación (registro de actividades de tratamiento, evaluaciones de impacto, contratos de encargo, política de privacidad), lo que habilita el informe de brechas documentales ya especificado (`GET /documents/required-gaps`). El módulo contempla poblar equivalencias con ISO 27001 —cuyos controles de Anexo A sostienen buena parte de las medidas técnicas y organizativas del artículo 32— para que la matriz de cobertura demuestre la reutilización de políticas y evidencias. La propia documentación de la plataforma ofrece material de referencia: el diseño del Trust Center aplica obligaciones GDPR concretas (base jurídica e información al interesado en formularios, retención de 24 meses con borrado automatizado, reparto responsable/encargado, atención a derechos de acceso y supresión), que ilustran el tipo de requisitos que el módulo debe modelar para los clientes. La interfaz debe comportarse correctamente para un marco sin catálogo de controles certificable, sustituyendo la SOA por el índice de información documentada, siguiendo la bifurcación `has_control_catalogue` prevista en la especificación.

Componentes

Se contempla: catálogo de artículos y obligaciones GDPR con carga masiva idempotente; `FrameworkScope` por organización con declaración de alcance; equivalencias curadas con ISO 27001; informes de brechas y cobertura por cláusula; integración con los módulos de documentos, evidencias, escalation y CAPA para gestionar incumplimientos; y presentación en el Trust Center como cumplimiento regulatorio, nunca como certificación.

Tecnología

Stack real de Dani Platform: backend FastAPI (Python) modular bajo `/api/v1` con modelos Pydantic; Google Cloud Firestore con agregados materializados mediante Cloud Functions; Google Cloud Storage con URLs firmadas; Keycloak (JWT, roles); contenedores Docker desplegados en Cloud Run; frontend React con tokens de diseño CSS, tema claro/oscuro e i18n.

Dani App Scanner

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: Dani App Scanner

Objetivo de negocio

Dani App Scanner es una plataforma de revisión de seguridad de código que sistematiza la detección de vulnerabilidades en el código de aplicaciones, sus dependencias y su configuración de infraestructura. Nace de una metodología propia de revisión de seguridad (v1.1) que define cinco fases de trabajo y diez áreas de principio —inyección, autenticación, criptografía, dependencias, secretos, control de acceso, errores y registro, seguridad de memoria, infraestructura como código y lógica de negocio—, cada una cruzada con OWASP Top 10 (2021), identificadores CWE y controles del Anexo A de ISO/IEC 27001:2022. El objetivo de negocio es que la evidencia de la revisión de código alimente directamente las actividades de cumplimiento y auditoría: cada hallazgo queda trazado al lenguaje de marcos que auditores y responsables de riesgo esperan, y la plataforma puede generar un paquete de evidencias ISO 27001 agrupado por control del Anexo A.

Cómo funciona

El flujo sigue las cinco fases de la metodología. En el alcance (Scoping) se registran repositorios, ramas, lenguajes, exposición y clasificación de datos, y se seleccionan las áreas de principio aplicables. La línea base automatizada (Baseline) ejecuta cuatro capas de escáneres —SAST, SCA, detección de secretos e IaC— produciendo un conjunto inicial de hallazgos y un inventario tecnológico. La revisión manual profunda recorre las rutas críticas siguiendo los checklists de las diez áreas, trazando el flujo de datos desde cada punto de entrada hasta sus sinks. El triaje confirma o descarta cada hallazgo, elimina falsos positivos y asigna severidad. Finalmente se reporta con evidencia, referencias de estándares y recomendaciones concretas, se acuerdan responsables y SLA de remediación por severidad, y se verifica el cierre. Una capa de IA opcional asiste el triaje (clasifica y redacta borradores de impacto y recomendación) sin auto-confirmar nunca un hallazgo, manteniendo a la persona en el bucle.

Componentes

Motor de ejecución de escaneos con checkpoints de pausa/cancelación y almacenamiento de artefactos (SBOM, salida cruda, logs); adaptadores conectables a herramientas reales (Semgrep, GitLeaks, Checkov, OSV-Scanner, Trivy) con CodeQL simulado; registro de hallazgos con máquina de estados (open → in_progress → fixed → verified, o accepted_risk/false_positive), historial de auditoría y detección de duplicados entre herramientas; checklists por área con criterios de salida; mapa de estándares OWASP/CWE/ISO; panel con KPIs y patrones sistémicos; ajuste de reglas, supresiones y reglas personalizadas; y generación de reportes (engagement, resumen ejecutivo, plan de remediación y paquete ISO en zip).

Tecnología

Stack real según la especificación de backend v1.0 y el README: FastAPI (Python) con SQLAlchemy 2.x y Pydantic v2, PostgreSQL como base de datos (enums nativos, uuid, timestampz, migraciones Alembic), autenticación OAuth2 password flow con JWT RS256 y RBAC de tres roles (Lead, Reviewer, Auditor), errores RFC 7807 con trace_id, concurrencia optimista ETag/If-Match, worker de trabajos en segundo plano con ARQ sobre Redis (con fallback en proceso), enrutador LLM de tres capas para el triaje asistido, y despliegue con Docker Compose. Contraseñas con Argon2id y credenciales de integración cifradas con Fernet.

Dani ETH

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Dani ETH

Objetivo de negocio

Dani-ETH es la plataforma de ethical hacking automatizado de Alloxentric, cuya misión es democratizar las pruebas de penetración de alto nivel, tradicionalmente caras, esporádicas y dependientes de talento humano escaso. La propuesta es una vigilancia constante que descubre y ayuda a remediar vulnerabilidades de forma continua, reduciendo la ventana de oportunidad de los atacantes y liberando a los expertos para el trabajo de arquitectura estratégica. Dentro de Alloxentric, Dani-ETH actúa como guardián de la integridad operacional: cuando detecta una vulnerabilidad de alta prioridad dispara flujos de comunicación proactiva hacia los equipos responsables y, en casos específicos, puede iniciar protocolos de parcheo automático. Al tratarse de un proyecto de continuidad, el equipo no parte de cero: recibe un "Resource Pack" con el plan completo de seis sprints, el código base y la configuración de contenedores, y su labor es elevar los wireframes actuales a componentes de grado de producción y añadir nuevos agentes.

Cómo funciona

Dani-ETH no es una simple interfaz sobre herramientas de seguridad, sino un orquestador inteligente. Integra de forma nativa herramientas líderes de la industria —Nmap, Nuclei, SQLMap, XSSStrike, JWT_Tool y Arjun— y les añade un componente de decisión autónomo: la IA evalúa los resultados de cada fase, desde el reconocimiento inicial hasta el escaneo de vulnerabilidades, y decide lógicamente cuál es el siguiente paso más eficiente. Así, una serie de scripts aislados se transforma en un flujo cohesivo que identifica fallos, prioriza el riesgo mediante un risk score y genera planes de remediación automáticos, cerrando el ciclo de seguridad de principio a fin. El trabajo de continuidad se centra en completar módulos críticos —en particular el Vulnerability Hub y el Patch Management—, refinar los criterios de aceptación de cada ticket, actualizar el manual de usuario con evidencias reales de funcionamiento y generar el manual de despliegue técnico, dentro de una metodología Scrum de seis sprints con una Definition of Done exigente (revisión por pares, cobertura de tests superior al 80%, staging y aprobación de QA).

Componentes

Dashboard de seguridad en tiempo real con risk score, heatmaps y widgets vía WebSocket; Vulnerability Hub para gestión centralizada de vulnerabilidades por categorías; motor de IA de pentesting con decisiones autónomas; gestión de parches (Patch Management) con algoritmos de distribución; gestión de equipos (carga, rendimiento, disponibilidad); y sistema de reportes ejecutivos, técnicos y operacionales en PDF, Excel y PowerPoint.

Tecnología

Stack real según la descripción del proyecto y el plan de sprints: frontend React 18 con TypeScript, Tailwind CSS, Socket.io-client, Axios y React Router v6; backend FastAPI (Python) —con Node.js/Express contemplado como alternativa—; PostgreSQL 15 como base de datos; Redis para caché y colas; Socket.io para comunicación en tiempo real; RabbitMQ/Celery para jobs en segundo plano; autenticación JWT con refresh tokens y bcrypt; y todo el ecosistema orquestado con Docker. Las herramientas de seguridad integradas son SQLMap, Nuclei, Nmap, JWT_Tool, Arjun y XSSStrike.

Directa v2

Tipo: Desarrollo · · Modalidad: Continuidad · Carpeta: Directa Asistente IA para directores de empresa

Objetivo de negocio

Un director de empresa recibe cada mes centenares de páginas de informes de comités, reportes de gestión, alertas regulatorias y síntesis de auditoría: leerlas todas es imposible e ignorarlas es irresponsable. DIRECTA es una plataforma de inteligencia directiva que procesa automáticamente toda la documentación que llega al directorio, la analiza con criterio crítico, detecta omisiones, contradicciones y riesgos, y entrega al director comprensión profunda, preguntas calibradas por actor y visión estratégica antes, durante y después de cada sesión. Su promesa central de seguridad se apoya en una arquitectura de tenant exclusivo por cliente, acceso biométrico indelegable, cifrado de extremo a extremo y procesamiento sin retención de los documentos originales.

Al ser un proyecto de continuidad, la versión 1.0 ya está en producción: app móvil para directores, panel de administración web para la secretaria del directorio, dos agentes de inteligencia (Interno y Competitivo) y el modelo de tenant exclusivo. La v2.0 avanza en dos ejes: estabilización a nivel enterprise (calidad funcional, performance, calidad analítica del pipeline de IA, UX, brand alignment y observabilidad 24/7) y especialización mediante catorce agentes de inteligencia sectorial.

Cómo funciona

El Agente Interno opera por eventos: la secretaria reenvía cada informe a una casilla de correo dedicada; el sistema lo clasifica por comité, período, tipo de riesgo y urgencia, genera el resumen crítico, las falencias detectadas y las preguntas estratégicas, y notifica al director en su app. El Agente Competitivo monitorea en segundo plano fuentes públicas (prensa, reguladores, competidores, indicadores macro) con lógica de relevancia editorial. En v2.0, los catorce agentes especializados (Financiero, Riesgos, Ciberseguridad, Legal, RRHH, Operaciones, Tecnología, Estrategia, ESG, entre otros) aplican lentes verticales sobre los datos ya procesados y se intercambian señales a través de un bus cross-agente, con los agentes Estrategia y Riesgos actuando como integradores transversales. El director sigue viendo una experiencia integrada, nunca catorce informes paralelos.

Componentes

App móvil del director (brief, alertas, preguntas calibradas, modo sala, seguimiento de compromisos), Admin Web Panel para la secretaria (usuarios, comités, documentos, sesiones, grabaciones, planes), pipeline de ingesta y análisis con IA, agentes horizontales (Interno y Competitivo), catorce agentes verticales con bus de señales, y arquitectura de tenant exclusivo con auditoría de accesos.

Tecnología

Según la documentación, la app móvil está construida en Flutter con acceso biométrico; el backend es TypeScript serverless sobre Supabase/Postgres con RLS (Row Level Security), con almacenamiento en S3 y transcripción de grabaciones vía Speechmatics. La v2.0 formaliza eval suites por agente, suite E2E con Playwright en CI, observabilidad con Sentry y CloudWatch/Datadog, sistema de íconos Lucide y design system documentado.

Dossier

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Dossier Project

Objetivo de negocio

Project Dossier (sistema Dossier/TraceLens) es un asistente de investigación automatizado integrado al calendario del usuario: detecta reuniones externas próximas y genera de forma autónoma un "Intelligence Brief" sobre los participantes, agregando datos de redes profesionales, registros corporativos gubernamentales (Companies House, SEC EDGAR) y prensa. El objetivo de negocio es reducir el tiempo de preparación previa a una reunión de ~20 minutos a menos de 30 segundos de esfuerzo activo, democratizando el nivel de briefing que suele reservarse a ejecutivos C-Suite con asistentes dedicados, con un modelo freemium por créditos (planes Free, Pro y Enterprise) y foco inicial en UK, USA y Europa continental.

Como proyecto de continuidad, ya existe una base documental consolidada: PRD v1.1 (2023), Manual de Usuario v1.0, prototipo HTML del dashboard y el documento de Especificaciones v2.0 (marzo 2026), que inventaría brechas técnicas y de negocio pendientes (decisiones de proveedores de datos, schema de base de datos, protocolo WebSocket, prompts, precios). Las mejoras apuntan a cerrar esas brechas, corregir errores y completar el MVP: dashboard web, automatización de calendario y entrega por email.

Cómo funciona

El usuario (o la automatización de calendario, 15 minutos a 1 día antes de la reunión) solicita un dossier configurando módulos por checkboxes: Identidad (siempre activo, 1 crédito), Historial Corporativo (3 créditos) y Presencia en Medios/Deep (5 créditos). Un orquestador analiza la configuración y lanza en paralelo agentes especializados por fuente; los resultados crudos convergen en una capa de síntesis LLM que detecta discrepancias (por ejemplo, entre LinkedIn y registros oficiales), clasifica alertas de riesgo (crítico: insolvencias o fraudes; advertencia: prensa negativa), propone hasta tres ice breakers con fuente citada y arma la línea de tiempo profesional. El frontend recibe estados progresivos vía WebSocket con resultados parciales si un agente falla; el dossier se cachea 24 horas y se elimina (privacidad por diseño, GDPR).

Componentes

Dashboard web, orquestador de agentes (Agent_Identity, Agent_Corporate_UK/USA/EU, Agent_News, Agent_Social), capa de síntesis LLM con schema JSON estricto, automatización de Google Calendar y Outlook, entrega por email (Gmail, Outlook, SMTP), billing por créditos, RBAC y audit logging conforme a los estándares Alloxentric.

Tecnología

Stack documentado: frontend Next.js (React, TypeScript, TailwindCSS); backend FastAPI (Python) con API REST + WebSocket, JWT y multi-tenancy; agentes con LangChain/LangGraph; síntesis con GPT-4o (arquitectura RAG, alucinación <5%); PostgreSQL para persistencia y Redis para caché de 24 h y rate limiting; APIs de Companies House, SEC EDGAR, OpenCorporates, Proxycurl/Apollo y Tavily/SerpApi; Stripe para pagos; cifrado TLS 1.3 y AES-256.

Galactum equipo 1

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO, inspirado en el modelo de negocio de Lords Mobile: cada jugador gestiona una Astronave (su base móvil), produce recursos, entrena fuerzas, se une a alianzas y compite en un mapa persistente compartido. El producto es free-to-play y vive de la retención diaria y la monetización por temporizadores, pase de temporada y tienda. El Equipo 1 construye la Plataforma y el Backend Core: el Servidor Galáctico Maestro (SGM) y toda la infraestructura compartida sobre la que trabajan los otros equipos. Es un equipo de servicio cuyo backlog se prioriza por las dependencias ajenas; la estrategia lo define como "el equipo que nunca debe ser cuello de botella". Su valor de negocio es directo: sin ledger confiable no hay economía vendible, y un bug aquí significa duplicación de recursos.

Cómo funciona

La arquitectura separa dos mundos: el SGM (monolito modular en NestJS, estado persistente, API REST /v1) y los Servidores de Sistema Solar (SSS, procesos Godot headless con estado efímero en memoria y snapshots a Redis cada 30 segundos). El cliente habla con el SGM por HTTPS REST y con el SSS por WebSocket; el SGM envía comandos al SSS por Redis pub/sub y el SSS reporta hechos consumados por REST interno con token de servicio. La regla de oro anti-cheat es transversal: el cliente nunca informa resultados, solo intenciones; toda cantidad se calcula server-side. El registro crea cuenta, jugador, salas iniciales y saldos de arranque en una única transacción; el login devuelve JWT (access de 15 minutos más refresh revocable) junto con la instancia SSS asignada por el SystemAllocator.

Componentes

Autenticación y cuentas (JWT, tablas accounts/players, guard global); Ledger Económico Transaccional (balances más ledger_entries append-only, EconomyService.apply con locks, idempotencia y rollback si un saldo queda negativo); Servicio de Temporizadores con lazy completion y reduce/finishNow para aceleraciones; Orquestación de instancias SSS (registro, heartbeat cada 10 s, asignación de jugadores); Pipeline de assets con manifiesto versionado y CDN; Telemetría por Redis Streams con esquema estándar de eventos; Anti-cheat, rate limiting y auditoría administrativa; y Configuración remota con feature flags (config_documents con ciclo draft/staged/live), base del backoffice del Equipo 5.

Tecnología

NestJS (TypeScript) sobre PostgreSQL 16 y Redis (pub/sub, streams, caché, rate limiting con sliding window). Servidores SSS en Godot headless desplegados como contenedores. JWT con JWKS interno para validación offline en los SSS; IDs UUIDv7, timestamps UTC ISO-8601, cantidades en BIGINT (nunca float); bucket S3-compatible más CDN para assets; API versionada bajo /v1 con errores estables {error_code, message}.

Galactum equipo 2

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO free-to-play, modelado sobre las mecánicas de retención y monetización de Lords Mobile: bases móviles (Astronaves), producción de recursos, héroes coleccionables, combate en un mapa persistente y eventos continuos. El Equipo 2 es dueño de "Nave, Economía y Progresión": todo el juego interior de la Astronave, donde el jugador pasa la mayor parte de su tiempo. La estrategia lo describe como el corazón del MVP, y su rol de negocio es la retención por progresión: salas que subir, elementos que producir y canjear, tecnologías que investigar, 45 tripulantes que coleccionar y fuerzas que entrenar. El rol de Game Designer-Economista asociado a este dominio se considera el más crítico del proyecto: si la curva de progresión falla, el juego no retiene.

Cómo funciona

El principio rector es "todo es catálogo más instancia": cada sistema tiene un catálogo estático versionado (editado vía `config_documents` del Equipo 1 desde el backoffice) y tablas de instancia por jugador; ningún número de balance se escribe en código. El Puente de Mando limita el nivel del resto de salas; mejorar una sala valida prerequisites y cola, debita recursos por el ledger y arranca un timer. La producción usa acumulación lazy (se calcula lo producido al consultar, sin jobs por jugador) y el canje de elementos es siempre 5→1 hacia arriba, sin canje inverso. Toda fuente de progreso (salas, techs, tripulantes, equipos, títulos) emite modificadores numéricos hacia un sistema único de modifiers, cacheado en Redis, que es además el contrato con el combate: el Equipo 3 congela un combat-snapshot del jugador al despegar cada flota.

Componentes

Salas de la Astronave con colas de mejora y Ayuda de Alianza; producción y canje de los 10 elementos (Fábrica y Segunda Plataforma) con Bóveda Cuántica; Investigación del Laboratorio (árbol DAG de 5 ramas); Sistema de Modificadores con recompute y snapshot de combate; Tripulantes (45 héroes con grados por Insignias, niveles por EXP, rangos y especialización irreversible a nivel 10); Armería (150 elementos de equipamiento, 90 equipos × 5 niveles, nanochips y loadouts intercambiables); entrenamiento de las 5 fuerzas en `troop_stacks` con Poder Galáctico; Criosantuario (flujo 80/20 de bajas, cola de reanimación 70/30 y API de liberación para las Operaciones de Rescate del Equipo 4); y la previsión de Bio-Símbiontes para Fase 2.

Tecnología

Módulos NestJS (TypeScript) dentro del SGM, sobre PostgreSQL y los servicios del Equipo 1 (EconomyService, TimerService, config/flags, telemetría); caché de modifiers en Redis. Cliente en Godot: toda la UI interior de la nave (salas, inventario, Armería, tripulación). Catálogos data-driven publicables: `catalog.rooms`, `catalog.elements`, `catalog.research`, `catalog.crew`, `catalog.equipment` y `catalog.troops`.

Galactum equipo 3

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO free-to-play, construido sobre el modelo de Lords Mobile: cada jugador desarrolla su Astronave, extrae recursos, forma alianzas y compite en un mundo persistente. El Equipo 3 es dueño de "Mundo y Combate": el mapa del sistema solar en tiempo real y el sistema de combate completo. La estrategia lo señala como el equipo de mayor riesgo técnico del proyecto (su perfil de referencia, Tech Lead de Godot multiplayer, es el más escaso del mercado). Su valor de negocio es la capa competitiva: el PvP, la minería disputada y la caza PvE dan sentido a toda la progresión que venden los demás equipos.

Cómo funciona

Rige un reparto estricto: lo persistente y económico vive en el SGM; lo efímero y espacial en el SSS (Servidor de Sistema Solar), un proceso Godot headless por sistema. El SSS simula un grid de 600×600 con spatial hash y loop a 2 ticks/s; las flotas se modelan por (origen, destino, timestamps) y la posición se deriva del tiempo, lo que abarata la simulación y hace que los vuelos sobrevivan a reinicios. El cliente se conecta por WebSocket, envía intenciones (nunca posiciones) y solo recibe deltas de su área de interés (~50 casillas). La Astronave viaja con una máquina de estados ANCHORED→TRAVELING→ANCHORED, con costo de plasma y reglas anti-dodge. Cuando dos flotas entran en contacto, el SSS delega en el resolver de combate del SGM: una función determinista, pura y versionada que aplica escudos, rondas con matriz de ventaja (+30 % por triángulo), reparto de heridos/muertos hacia Bahía Médica y Criosantuario, y saqueo limitado por la Bóveda Cuántica.

Componentes

Servidor SSS en Godot (simulación, snapshots a Redis cada 30 s, recovery); protocolo cliente-SSS por WebSocket con validación JWT vía JWKS; anclaje, viajes y Saltos Warp; minería de asteroides nivel 1-5 con ocupación exclusiva y tile hitting; sensores y espionaje con precisión por nivel de Sala de Sensores; el resolver de combate con Asedios (rally con herencia de boosts del líder) y reportes de batalla; PvE de Xenofórmulas (24 especies × 5 niveles, costo de Energía, bono por caza consecutiva) y Colmenas; y una estrategia de testing obligatoria: TDD del resolver, property-based testing, harness de balance de 10.000 combates y soak tests con 500 bots concurrentes.

Tecnología

Godot headless (GDScript/C#) para el SSS con export web compatible; WebSocket con JSON (MessagePack como opción); Redis para snapshots y pub/sub; resolver de combate como módulo NestJS en el SGM sobre PostgreSQL, integrado con el ledger, los combat-snapshots del Equipo 2 y CryoService; spawns y tasas data-driven desde config_documents.

Galactum equipo 4

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO free-to-play que replica en clave espacial la arquitectura de Lords Mobile: bases (Astronaves), producción, héroes, combate en mapa persistente y una capa social intensa. El Equipo 4 es dueño de "Social y Política": su misión declarada es convertir jugadores individuales en miembros de algo — Alianzas, cooperación diaria, chat y, más adelante, el endgame político. Su métrica de negocio es la retención a 30 o más días: la pertenencia a una alianza es el mayor factor de permanencia en el género, y las Cápsulas de Suministro convierten el gasto de los grandes compradores en recompensas para toda la alianza, sosteniendo el ecosistema free-to-play.

Cómo funciona

Las alianzas tienen rangos R1-R5 con permisos definidos como datos (matriz `role_permissions` en catálogo, nunca hardcodeada), búsqueda con filtros y tres modos de ingreso (abierto, solicitud, invitación). El ritual diario es la Ayuda Mutua: cada clic de un compañero reduce un 1 % (mínimo 1 minuto) un temporizador ayudable y otorga 800 Bonos de Alianza al ayudador, con controles anti-abuso. Las Cápsulas de Suministro —generadas por cada compra o xenoforma cazada de un miembro— usan un modelo sin fan-out de escritura: crear una cápsula es un INSERT y "abrir todas" es un batch con un único apunte al ledger. El chat corre en un gateway WebSocket propio, separado del socket del SSS, y difunde eventos de coordinación (asedio abierto, miembro atacado, cápsula legendaria), convirtiéndolo en la sala de guerra del juego. El Festival de la Flota (evento cooperativo semanal) y las Operaciones de Rescate del Criosantuario se evalúan consumiendo la telemetría del Equipo 1 de forma idempotente, sin instrumentar cada feature.

Componentes

Núcleo de Alianzas (modelo, permisos, búsqueda, ingreso y casos borde como la transferencia de liderazgo); Ayuda Mutua; Bonos, Tienda de Alianza y Tesorería con límites diarios por rol y auditoría por ledger; Cápsulas de Suministro con expiración a 24 h y cola máxima de 100; chat y gateway de tiempo real con moderación; Festival de la Flota (tableros, misiones y marcador colectivo); Operaciones de Rescate (las misiones PvE que liberan el 30 % del pool del Criosantuario, jugables en grupo); y la previsión del endgame político de Fase 2+: títulos, Portal de Salto, Plataformas Orbitales y Guerra de Sistemas.

Tecnología

Módulos NestJS (TypeScript) en el SGM sobre PostgreSQL (índice GIN para búsqueda, claves compuestas para claims) más un gateway WebSocket propio en NestJS con canales por alianza, sistema y mensajes directos; Redis para los últimos 200 mensajes por canal y rate limiting (1 msg/s); ledger con `owner_type player|alliance` (Equipo 1); UI en Godot con virtualización de listas para 100 miembros, requisito explícito del módulo.

Galactum equipo 5

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO free-to-play, diseñado como equivalente espacial de Lords Mobile, cuyo modelo de negocio se basa en temporizadores acelerables, pase de temporada, tienda segmentada y un calendario de eventos que sostiene el hábito diario. El Equipo 5 es dueño de "LiveOps, Monetización y Backoffice": opera el juego como servicio. La estrategia es explícita: el calendario de eventos superpuestos ES el producto a largo plazo (la referencia documenta que el pase de batalla duplicó los ingresos mensuales de Lords Mobile). Este equipo se forma en Fase 3, pero el backoffice básico lo encarga al Equipo 1 desde Fase 0. Su definition of done resume el objetivo: operar eventos, pase, ofertas y compensaciones, medibles en dashboards, sin ningún deploy.

Cómo funciona

El motor de eventos es data-driven: configurar un evento (Órdenes del Día cada 3 horas, Eventos de Singularidad de fin de semana) es editar un documento, no escribir código. Un consumer de la telemetría del Equipo 1 mapea eventos de juego a puntos según reglas de la definición activa, con leaderboards en Redis por bracket de nivel de Puente y kill switch de cancelación inmediata; el calendario valida que siempre haya al menos 2 eventos activos. El Pase de Expedición (temporada de 30 días, pista gratuita y premium de 50 niveles) reutiliza el mismo scorer; la compra premium desbloquea retroactivamente con claim explícito. El flujo IAP es crítico en seguridad: el recibo de Google/Apple se valida server-to-server y la entrega usa el transaction_id como clave de idempotencia (un recibo jamás entrega dos veces), emitiendo purchase_completed hacia las cápsulas del Equipo 4. Los speedups con Materia Oscura calculan costo por tiempo restante con revalidación en servidor.

Componentes

Motor de eventos con calendario por capas; Pase de Expedición; tienda, ofertas segmentadas e IAP con contabilidad de Materia Oscura comprada/ganada/gastada (requisito regulatorio); Licencia de Comandante (VIP de 15 niveles con beneficios como modifiers); Expediciones (campaña PvE, fuente diaria de Insignias, con modo Élite y auto-batalla en Licencia 6); backoffice web de administración (editores de catálogo con draft/staged/live, diff y rollback, búsqueda de jugador, bans, compensaciones masivas con dry-run, moderación); y analítica de producto: retención D1/D7/D30, conversión, ARPU/ARPPU, embudo de onboarding y balanza sources/sinks de la economía.

Tecnología

Módulos NestJS en el SGM más backoffice en React con API admin separada (2FA y roles admin); PostgreSQL y Redis (sorted sets para leaderboards); config_documents y feature_flags del Equipo 1 con A/B testing por bucketing estable; validación server-to-server contra las APIs de Google/Apple; vistas materializadas sobre la telemetría con Metabase/Superset en réplica de lectura.

Galactum Equipo 6

Tipo: Diseño · Alumnos: 3 · Modalidad: Nuevo · Carpeta: Galactum

Objetivo de negocio

Galactum es un juego de estrategia espacial multijugador tipo 4X/MMO free-to-play, equivalente espacial de Lords Mobile: Astronaves como bases móviles, producción de recursos, tripulantes coleccionables, alianzas y combate en un mapa persistente. En este género la identidad visual es un activo de negocio directo: el coleccionismo (tripulantes, equipos, xenofomas) solo retiene si los assets son atractivos y distinguibles, y la propia referencia del proyecto documenta que Lords Mobile llegó a cambiar de motor gráfico para mantener su relevancia visual. El Equipo 6, de perfil de Diseño, se propone como el equipo de "Renderización y Assets": cubrir la necesidad que la Estrategia de Desarrollo asigna a un pool transversal de arte (un art director y 2-3 artistas 2D/UI), dado que el catálogo visual del juego es enorme y ningún equipo de producto lo produce por sí mismo.

Cómo funciona

Se contempla que el equipo trabaje como pool de servicio con una cola de pedidos priorizada por los Product Owners de los equipos de producto, según describe la estrategia general. El flujo previsto abarca definir la dirección de arte del universo Galactum, producir los assets 2D/UI que consumen las pantallas del cliente Godot (la home de la nave del Equipo 2, el mapa 600×600 del Equipo 3, el módulo de Alianzas del Equipo 4, la tienda y eventos del Equipo 5) y entregarlos por el pipeline de assets del Equipo 1: los assets de catálogo no se empaquetan en el binario, sino que se publican en un bucket con CDN y un manifiesto versionado que el cliente descarga bajo demanda y cachea. Se propone además atender la renderización en el cliente: theming de la UI en Godot, representación de las entidades del mapa e interpolación visual del movimiento que simula el servidor.

Componentes

Se contemplan como entregables: la guía de estilo y dirección de arte del proyecto; el catálogo de retratos de los 45 tripulantes; las 24 xenofomas con sus 5 niveles; los 90 equipos de la Armería y sus iconografías; las ~15 salas de la Astronave y las naves; iconos de los 10 elementos, items y monedas; los kits de UI (paneles, botones, listas, mapas) para las pantallas de todos los equipos; y la organización de assets con `asset_id` referenciables desde los catálogos data-driven.

Tecnología

Se propone trabajar sobre el stack real del proyecto: cliente Godot (Control nodes y theming para UI; escenas 2D del mapa), con exportación web contemplada; herramientas de ilustración y edición 2D a criterio del equipo; y entrega vía el pipeline del Equipo 1 (bucket S3-compatible, CDN, manifiesto JSON versionado con checksums sha256 generado en CI), de modo que cada asset quede identificado y versionado para los catálogos de los equipos 2 y 5.

Hydro-Sports

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Hydro - sports

Objetivo de negocio

Hydro Precision es la aplicación companion para las prendas deportivas de Hydro con sistema BFR (Blood Flow Restriction, restricción del flujo sanguíneo), una técnica de entrenamiento de alto estrés metabólico y baja carga mecánica. La app cierra la brecha entre una fisiología compleja y la usabilidad de consumo cumpliendo tres funciones: refuerzo de seguridad (verificar el uso correcto de la prenda y descartar contraindicaciones médicas), optimización del entrenamiento (selección de ejercicios a partir de la anatomía) y seguimiento holístico (integración de datos sistémicos de salud desde wearables). El objetivo es reducir el riesgo y la responsabilidad legal asociados al mal uso del BFR, y convertir la prenda en un producto guiado, medible y seguro.

Cómo funciona

El onboarding actúa como "gatekeeper": un cuestionario médico obligatorio bloquea el uso ante contraindicaciones críticas (trombosis venosa profunda, hipertensión, embarazo) y el perfil físico captura la circunferencia de las extremidades, variable principal para calcular la presión de oclusión segura (calculadora LOP con fórmula tipo Loenneke). El usuario selecciona el grupo muscular en un mapa corporal interactivo y la app resalta la zona proximal segura para el cuff y recomienda ejercicios. El "Cuff Check" con cámara y realidad aumentada verifica que la prenda esté a la altura y tensión correctas, complementado con el test de rellenado capilar. Durante el entrenamiento, la visión por computador se ejecuta en el dispositivo (estrategia local-first/Edge AI): la app extrae landmarks esqueléticos, calcula ángulos articulares y corrige la forma en tiempo real (algoritmo anti-lockout, metrónomo de tempo, "ghost rep"). La integración con Apple HealthKit, Google Fit, Oura y Whoop contextualiza la carga: si el HRV es bajo o el sueño insuficiente sugiere una sesión ligera de recuperación, y alerta si la frecuencia cardíaca supera el 80% del máximo. Al finalizar, se sincroniza un payload JSON (no el video) con reps, tempo y presión, y se activa la ventana de nutrición post-entrenamiento.

Componentes

App móvil con mapa corporal 3D y HUD de cámara; backend REST con 42 endpoints en grupos de autenticación, perfil, screening médico, metas de entrenamiento, equipamiento, catálogo de ejercicios, sesiones, AR Cuff Check, readiness, wearables, analítica, calculadora LOP, notificaciones y sincronización offline.

Tecnología

La documentación evolucionó: el PRD y el TDD iniciales recomendaban Flutter con nube GCP (Cloud Run, Firestore, BigQuery, Cloud Storage, Vertex AI), mientras el Development Plan v2 (febrero 2026, "Horizon 1 — Goal-Driven") define React Native (Expo) con Zustand/Redux Toolkit, TanStack Query y react-native-svg; backend Node.js (Express/Fastify) en AWS ECS/Lambda o Cloud Run, PostgreSQL (TimescaleDB para métricas), Firebase Auth o Cognito con JWT, FCM para push, y pose estimation on-device con Google ML Kit/MediaPipe.

Inteligencia de Mercado

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Inteligencia de mercado Encipharm

Objetivo de negocio

Enci-Intel es la plataforma de inteligencia de mercado de Encipharm, empresa chilena con más de 25 años en salud, nutrición y bioseguridad animal que compite con multinacionales como Zoetis y Virbac y laboratorios nacionales como Drag Pharma y Agrovét. Hoy la inteligencia competitiva se recopila manualmente en planillas Excel, con latencias de 1 a 7 días para enterarse de cambios de precios, nuevos registros regulatorios o desabastos de competidores. La plataforma reemplaza ese proceso por agentes autónomos que monitorean 24/7 y entregan alertas accionables al equipo comercial y a la gerencia; su métrica estrella es el Tiempo de Respuesta Competitiva, con meta de menos de 4 horas en el MVP y menos de 2 horas a seis meses.

Como proyecto de continuidad, existe documentación funcional y PRD completos, una API especificada con 22 endpoints y un documento de alcance reducido firmado en mayo de 2026 que dejó la entrega en tres pilares: 2 agentes activos, dashboard funcional alineado al wireframe e integración de los endpoints críticos. Las mejoras apuntan a completar el alcance original: reincorporar el resto de los agentes y avanzar hacia la Fase 2 (Análisis de Productos con drill-down, Mapa Competitivo, nuevos competidores como Virbac, Elanco y Alcamed, reportes exportables).

Cómo funciona

Cloud Scheduler activa periódicamente cada agente del enjambre según su frecuencia configurada: el Agente de Vigilancia ISP detecta nuevos registros de productos veterinarios de la competencia; el Agente de Inteligencia Aduanera extrae Declaraciones de Importación del SAG y Aduanas; el Agente de Monitoreo Competitivo rastrea LinkedIn, portales agropecuarios y gremios; el Agente de Benchmarking de Precios compara precios por SKU y alerta cuando la brecha supera umbrales configurables (15% por defecto). El Motor de Alertas Estratégicas consolida todo y publica notificaciones priorizadas por criticidad (rojo/amarillo/verde) en el Dashboard, que se actualiza cada 60 segundos. Un Consultor Veterinario IA (chat con Gemini vía Vertex AI, entrenado sobre papers, fichas técnicas y protocolos) responde consultas clínicas con referencias, filtros por especie y streaming.

Componentes

Dashboard de Alertas, módulo de configuración y monitoreo de Agentes de Inteligencia, Consultor Veterinario IA (MVP), más Análisis de Productos y Mapa Competitivo (Fase 2), con roles Admin, Comercial y Gerencia.

Tecnología

Stack documentado: frontend React (TypeScript); backend FastAPI (Python 3.12); base de datos Firestore (GCP, NoSQL); agentes en contenedores serverless sobre Cloud Run, orquestados por Cloud Scheduler y Cloud Tasks; autenticación RBAC con Keycloak (OIDC/OAuth 2.0, JWT); IA con Vertex AI (Gemini) y embeddings; CI/CD con GitHub Actions hacia Cloud Run.

KalliCode equipo 1

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: KalliCode

Objetivo de negocio

KalliCode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código a través de tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable de toda la operación. El negocio vende capacidad de entrega —"líneas de producción" de software— a empresas con sistemas legacy, incluyendo despliegues soberanos on-premise para banca y sectores regulados.

Este equipo es dueño de **CodeMapping**, el núcleo defendible del producto: el grafo de conocimiento vivo del código, el modelo de datos, la ontología de negocio, la documentación y el historial de incidentes del cliente, que todo agente consulta antes de actuar. Si el grafo miente, el pipeline entero produce trabajo equivocado con aspecto correcto; por eso el plan trata la corrección y la trazabilidad del grafo como requisitos de seguridad. El plan cubre 12 sprints semanales (3 de agosto a 23 de octubre de 2026), con 70 tickets y 343 Story Points.

Cómo funciona

El subsistema ingiere el repositorio del cliente (sincronización git por rama y por tenant, catálogo de archivos con hash, análisis sintáctico con tree-sitter y refresco incremental por diferencia de conjuntos) y su base de datos (extracción de tablas, columnas y claves ajenas con salvaguardas de solo lectura). Sobre esa base determinista, una cadena semántica de cinco pasos con LLM genera descripciones de símbolos, consolida definiciones, induce la ontología de negocio, clasifica documentación y propone "nuggets" de contexto; nada de baja confianza entra al grafo sin pasar por la cola de curación humana. Por la decisión D18, el grafo abandona Neo4j y Qdrant y se reconstruye sobre PostgreSQL 16 + pgvector (esquema `cm`, clave compuesta por tenant, RLS forzado, carga idempotente, snapshots). El punto de partida es un prototipo de 775 líneas de un solo cliente que se reconstruye por capas hasta escalar a 200.000 nodos.

Componentes

Siete épicas: fundaciones del paquete, CI y observabilidad (ECM1); migración a PostgreSQL + pgvector (ECM2); ingesta determinista, escala y refresco incremental (ECM3); enriquecimiento semántico con LLM, con validación por JSON Schema y presupuesto de tokens (ECM4); capas de conocimiento y curación humana —documentación, nuggets, incidentes— (ECM5); capa de consulta e integración con la plataforma —módulo `cm\_queries` con impacto a k saltos, subgrafo y dependencias ocultas, contrato v1 para Backend y Testing, worker de ingesta— (ECM6); y rendimiento, escala y release GA (ECM7).

Tecnología

Python empaquetado con uv, tree-sitter para siete lenguajes (TypeScript, TSX, JavaScript, Python, Java, SQL; C#, Go, PHP y Kotlin en el Sprint 11), PostgreSQL 16 con pgvector y CTE recursivos (identificadores UUIDv5, índices HNSW), embeddings vía endpoint TEI con BGE-M3, pytest con repositorio de referencia, ruff/black/mypy, contenedores Docker y despliegue junto a API y workers.

Kallicode equipo 2

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Kallicode

Objetivo de negocio

Kallicode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código con tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable. El negocio vende capacidad de entrega por "líneas de producción" a empresas con sistemas legacy, con opción soberana on-premise para sectores regulados.

Este equipo es dueño de **Database**, el subproyecto del que dependen los otros cuatro: el esquema SQL es el contrato más consumido del programa —CodeMapping carga el grafo contra él, Backend deriva sus modelos, Portal muestra lo que él permite consultar y Testing prueba sus invariantes—. Es además donde vive la propiedad de seguridad central del producto (el aislamiento entre clientes) y su garantía probatoria (la cadena de hashes de auditoría). El plan cubre 12 sprints semanales (3 de agosto a 23 de octubre de 2026), con 72 tickets y 345 Story Points.

Cómo funciona

El punto de partida es el mejor de los cinco scaffolds: 41 tablas en tres esquemas lógicos (36 en `core`, 3 en `vec`, 2 en `audit`) que coinciden una a una con el documento de diseño, con triggers que implementan invariantes en el motor y la batería `core.validar\_integridad()` de veinte chequeos cruzados. Lo que falta es lo que un modelo multi-tenant exige: integridad referencial por tenant (claves ajenas compuestas con `tenant\_id`, hoy escalares), gobierno de cambios (ledger de migraciones `core.schema\_migrations` con runner y verificación de hash) y garantía de que el RLS forzado sobrevive a futuras migraciones (event trigger). A eso se suma corregir la verificación de la cadena de auditoría —que hoy no verifica el encadenamiento— y modelar en relacional el grafo de código que la decisión D18 trae desde Neo4j: 17.323 nodos y 23.309 aristas por cliente medio, más de 200.000 en despliegues completos, con presupuesto de latencia medido para los recorridos a k saltos.

Componentes

Siete épicas: gobierno del esquema y migraciones versionadas (EDB1); integridad multi-tenant y RLS endurecido (EDB2); integridad probatoria y tablas append-only (EDB3); esquema `cm` del grafo de código y capas vectoriales (EDB4); rendimiento —índices tenant-first, particionado, pgvector a escala— (EDB5); corrección del modelo comercial y de constraints, con cuota por aniversario de contratación (EDB6); y operación: retención con pg_cron, export WORM de auditoría, backup y recuperación ante desastre con RPO/RTO medidos, y ampliación de la validación a 26 chequeos (EDB7).

Tecnología

PostgreSQL 16 con pgvector (índices HNSW, halfvec, recall calibrado bajo RLS) sobre Azure Database for PostgreSQL Flexible Server; SQL puro con `WITH RECURSIVE ... CYCLE` (Apache AGE no está disponible); particionado con pg_partman y purgas con pg_cron; migraciones versionadas `db/migrations/NNN\_\*.sql`; roles `kallicode\_app`, `kallicode\_worker` y `kallicode\_admin`; CODEOWNERS, linter de bloqueos peligrosos y benchmarks en CI.

KalliCode equipo 3

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: KalliCode

Objetivo de negocio

KalliCode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código con tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable. El negocio vende capacidad de entrega por "líneas de producción" a empresas con sistemas legacy, con despliegue soberano on-premise para sectores regulados.

Este equipo es dueño del **Backend**, el subproyecto que convierte el resto del programa en un producto: el esquema guarda el estado, el grafo aporta el conocimiento y el portal muestra lo que ocurre, pero es el backend quien decide qué ocurre —recibe el ticket, lo clasifica, elige el modelo, ejecuta la etapa, exige la firma humana, descuenta la cuota y sella la evidencia—. Es también el único que toca las tres cosas que un cliente no perdona: el aislamiento entre organizaciones, las credenciales de su repositorio y el gasto en modelos de lenguaje. El plan cubre 12 sprints semanales (3 de agosto a 23 de octubre de 2026), con 72 tickets y 332 Story Points.

Cómo funciona

El scaffold FastAPI trae 77 de los 79 endpoints especificados en 15 routers, con tres superficies: `/api/v1` para el portal, `/api/v1/webhooks` para integraciones entrantes y `/internal/v1` para el pipeline. Los cinco primeros sprints cierran 25 hallazgos bloqueantes y altos (RLS inoperante, fugas cross-tenant, idempotencia ignorada, secretos en texto plano en Redis). La ausencia mayor es el orquestador: el bucle que reserva jobs, resuelve la etapa (Triage → Design → Build → QA → Security → Deploy), ejecuta sus pasos con las 21 plantillas del catálogo LLM, publica artefactos por la API interna y transiciona por la máquina de estados única. El motor LLM implementa tres tiers con escalado flash → pro → fable, retroceso con jitter, interruptor de circuito, contabilidad de tokens por tenant y mitigación de inyección de prompts.

Componentes

Ocho épicas: fundaciones —RLS efectivo, empaquetado y despliegue— (EBE1); identidad, sesiones y RBAC con revocación de tokens (EBE2); orquestador del pipeline y máquina de estados (EBE3); motor LLM con tiers, escalado y contabilidad (EBE4); gates, QA y seguridad de código (EBE5); workers resilientes y ciclo comercial (EBE6); auditoría con hash encadenado, exportación, bóveda de secretos e integraciones (EBE7); y observabilidad, endurecimiento y release (EBE8).

Tecnología

Python en workspace uv (`packages/core`, `apps/api`, `apps/workers`), FastAPI con Pydantic, PostgreSQL 16 con sesiones RLS por rol, Redis para colas (consumo at-least-once, cola de mensajes muertos, elección de líder), Argon2id y JWT RS256 con denylist de jti, bóveda de secretos sobre Azure Key Vault, Azure Blob Storage con firmas, imágenes Docker multi-etapa sin privilegios, red privada con mTLS para la API interna, `/metrics` y log estructurado con trace_id, tenant_id y actor.

KalliCode equipo 4

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: KalliCode

Objetivo de negocio

KalliCode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código con tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable. El negocio vende capacidad de entrega por "líneas de producción" a empresas con sistemas legacy, con modalidad soberana on-premise para sectores regulados.

Este equipo es dueño del **Portal**, la única superficie por la que el cliente ve KalliCode: todo lo que los demás subproyectos construyen —el grafo, el aislamiento, el pipeline, la evidencia de las pruebas— existe para el cliente solo en la medida en que este portal lo muestre. Un defecto de backend se traduce en un dato equivocado; un defecto de portal, en un producto que parece roto. El plan cubre 12 sprints semanales (3 de agosto a 23 de octubre de 2026), con 73 tickets.

Cómo funciona

El portal es una aplicación React con TypeScript que reproduce las nueve pantallas del wireframe del cliente (tablero, tickets, aprobaciones, auditoría, onboarding, modelos y consumo, más detalle de ticket, expediente y login), con tokens de estilo, modo claro/oscuro y tres idiomas. El punto de partida es un esqueleto: de las 364 claves de interfaz del wireframe existen 122 y se consumen 32 de los 62 endpoints disponibles. Los cinco primeros sprints cierran los 16 hallazgos bloqueantes y altos —la renovación de sesión sin exclusión mutua que expulsa al usuario de todos sus dispositivos, tokens en sessionStorage, ausencia de ErrorBoundary, RBAC de interfaz inexistente— y ponen la base de datos, accesibilidad y responsive. Del sexto al décimo se construyen las siete pantallas que faltan, con el asistente de onboarding de cinco pasos como bloque más crítico para el negocio.

Componentes

Seis épicas: fundaciones del frontend —tooling, CI, pruebas y resiliencia— (EPT1); sesión, seguridad del cliente y RBAC de interfaz con mapa ruta→roles (EPT2); capa de datos, contrato y validación en frontera (EPT3); pantallas y flujos pendientes, el mayor bloque de trabajo nuevo (EPT4); responsive desde 375 px, accesibilidad WCAG 2.1 AA e internacionalización en español, inglés y portugués (EPT5); y rendimiento, observabilidad y release (EPT6). El equipo posee dos contratos públicos: los tokens de estilo con el sistema de componentes base, y la convención `data-testid` que consume la suite E2E de Testing.

Tecnología

React con TypeScript en modo `tsc --strict` y build con Vite; TanStack Query como capa única de estado de servidor (caché, deduplicación, cancelación, invalidación por mutación); esquemas zod en la frontera y tipos generados desde el OpenAPI publicado; ESLint (react-hooks, jsx-a11y), Prettier, husky y lint-staged; Vitest con Testing Library y dobles de API con msw; auditoría de accesibilidad con axe; i18n con plurales y formateadores por locale; carga diferida por ruta y telemetría de errores de cliente.

KalliCode equipo 5

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: KalliCode

Objetivo de negocio

KalliCode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código con tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable. El negocio vende capacidad de entrega por "líneas de producción", con opción soberana on-premise para sectores regulados donde la evidencia y la auditabilidad son criterio de compra.

Este equipo es dueño de **Testing**, con responsabilidad doble fijada por la decisión D19. Primera: la etapa QA del pipeline de producto —el módulo que deriva pruebas ejecutables de los criterios de aceptación del gate 1, las ejecuta contra el sistema del cliente, produce evidencia inmutable y escribe la matriz de resultados sobre la que un aprobador humano firma el gate 2—. Segunda: la estrategia de pruebas y el CI/CD de la plataforma KalliCode misma. El punto de partida: cero pruebas y cero integración continua en todo el repositorio, causa raíz común de los defectos bloqueantes de los demás subproyectos. El plan cubre 12 sprints (3 de agosto a 23 de octubre de 2026), con 72 tickets.

Cómo funciona

Los sprints 1 y 2 son el camino crítico del programa: el CI de los cinco proyectos y la suite de aislamiento multi-tenant generada desde el catálogo (dos sesiones con tenants distintos y aserción de no-fuga por tabla) son la evidencia del hito M1. Los sprints 3 a 5 cierran los 20 hallazgos bloqueantes y altos de `testing/` —el gate 2 que aprueba con cero pruebas ejecutadas, código generado por LLM ejecutado sin sandbox, ausencia de `tenant\_id`—. Del sexto al décimo se construye la etapa QA descrita en el documento técnico: las cuatro cadenas LLM (derivación de casos, resumen de resultados, checkpoint visual, diagnóstico de fallo), trazabilidad criterio↔test, evidencia con hash, cuarentena de pruebas inestables y reparación de selectores auditada con pull request.

Componentes

Ocho épicas: CI/CD y entornos de prueba (EQA1); unitarias e integración de la plataforma (EQA2); aislamiento multi-tenant y seguridad (EQA3); contrato de API, esquema y migraciones (EQA4); E2E del portal y accesibilidad (EQA5); motor de generación de pruebas de la etapa QA (EQA6); ejecución, evidencia y cuarentena (EQA7); y carga, resiliencia, UAT e informe de calidad del programa (EQA8), más los diez niveles de la pirámide de pruebas del capítulo 7.

Tecnología

Integración continua con contenedores de servicio (pgvector/pgvector:pg16, Redis, Azurite, Mailpit); pytest con polyfactory y testcontainers contra PostgreSQL 16 real —nunca SQLite—; schemathesis y oasdiff para el contrato OpenAPI; hypothesis para la máquina de estados; Playwright con axe-core como puerta de merge; Maestro para mobile; k6 para carga; gitleaks, semgrep y trivy; squawk como linter de migraciones; cobertura del 80 % global y 95 % en seguridad y auditoría (D17).

Kallicode equipo 6

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Kallicode

Objetivo de negocio

Kallicode es una plataforma SaaS multi-tenant que automatiza la producción de software mediante un pipeline orquestado por IA: recibe tickets del cliente, diseña alternativas, ejecuta cambios de código con tres tiers de modelos de lenguaje, aplica controles de calidad y seguridad, exige aprobaciones humanas (gates) y mantiene una auditoría inmutable. El negocio vende capacidad de entrega por "líneas de producción" a empresas con sistemas legacy, con modalidad soberana on-premise para sectores regulados.

Este equipo (E6) construye la **Consola de Administración**, el único flujo de valor que no atiende al cliente sino a Kallicode: su producto es la capacidad de la propia empresa para operar y vender lo que ha construido —saber cuántos clientes hay y qué consumen, ver si el pipeline va bien y responder una pregunta comercial sin abrir una consola de base de datos—. Es el único equipo que empieza sin una línea de código: hereda solo un wireframe HTML de 84 KB con siete pantallas maquetadas. El plan cubre 12 sprints (3 de agosto a 23 de octubre de 2026), con 62 tickets y 294 Story Points con rampa de capacidad.

Cómo funciona

La consola es la superficie más peligrosa de la plataforma: la única que mira a todos los clientes a la vez, sobre una arquitectura diseñada para que eso sea imposible. El plan lo resuelve sin bypass de RLS: un rol de solo lectura con políticas explícitas por tabla y columna, preferentemente sobre vistas agregadas; acceso a datos identificables justificado y temporal, con código y datos personales enmascarados por defecto; y toda lectura sellada en la cadena de auditoría del tenant afectado y visible para el propio cliente en su portal. Por eso el orden es inflexible: los cinco primeros sprints construyen identidad del personal, modelo de acceso, auditoría y la API `/admin/v1`; las pantallas no empiezan hasta el Sprint 6. Doce brechas de diseño (cuatro bloqueantes) se cierran con ADR aprobado más pieza implementada y probada.

Componentes

Seis épicas: fundaciones del repositorio `kallicode-admin` e identidad de personal con segundo factor obligatorio (EAD1); API `/admin/v1` y modelo de acceso cross-tenant (EAD2); auditoría y gobierno del acceso del personal (EAD3); consola de negocio —panel general, clientes, suscripciones y consumo de tokens— (EAD4); consola de operaciones —agentes del pipeline, modelos y proveedores, y acuerdos de nivel de servicio definidos con el Product

Owner— (EAD5); y endurecimiento, observabilidad y release (EAD6). Las siete pantallas: biz-overview, biz-customers, biz-subscriptions, biz-tokens, ops-agents, ops-models y ops-sla.

Tecnología

Repositorio nuevo con CI y despliegue independientes; backend Python con FastAPI y contrato OpenAPI propio versionado; PostgreSQL con vistas materializadas, worker de refresco y pool analítico separado para no degradar el portal; identidad de staff en realm separado con OIDC y segundo factor; frontend React con TypeScript y TanStack Query reutilizando los tokens y componentes del portal cliente; registro append-only de acciones del personal; sin superficie pública y con separación de red verificada.

ND Sales Assistant v2

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Sales Assistant NoonDalton

Objetivo de negocio

La NoonDalton AI Marketing Suite es una plataforma SaaS de marketing B2B con inteligencia artificial para NoonDalton, que cubre dos frentes: la generación de contenido de marketing para redes sociales (a partir de "Book Concepts" que derivan en whitepapers, one-pagers, posts sociales, infografías e imágenes IA) y la generación asistida de propuestas comerciales con exportación profesional y conexión al flujo de ventas. El objetivo es industrializar la producción de contenido y propuestas de calidad, conectando prospección, campañas y reportes en un solo sistema.

Como proyecto de continuidad, la versión 1 fue entregada y aceptada formalmente en julio de 2026 (producción en `marketgenai.vercel.app`) con nueve módulos operativos: Proposals (wizard de 3 pasos, 6 estados de ciclo de vida, score automático, export PDF/DOCX), Opportunities, Content Library, Outreach, AI Assistant, Campaigns (wizard de 7 pasos que conecta todos los módulos), Reports con KPIs reales, Book Concepts y Seguridad (RBAC, audit logs, OAuth). Las mejoras de v2 apuntan a lo que quedó explícitamente fuera del alcance aceptado: worker asíncrono Celery/Redis en producción (hoy hay fallback síncrono), migración de los módulos que aún usan Gemini hacia DeepSeek (analysis, chat legacy), pruebas automatizadas con pytest, one-pager con exportación PDF (WeasyPrint), feature gating premium y white-labeling, además de corrección de errores.

Cómo funciona

En el frente de contenido, el usuario crea un Book Concept con título, descripción y keywords; el LLM propone el índice de capítulos, genera el contenido por capítulo en paralelo (~30 páginas), permite refinarlo con instrucciones dirigidas y editarlo, y deriva activos de marketing por plataforma que pueden publicarse en redes sociales conectadas por OAuth o traducirse a otros idiomas. En el frente comercial, las propuestas se construyen con plantillas con placeholders y líneas de servicio; el LLM genera el borrador final —enriquecido vía RAG con las cotizaciones históricas de Netprovider integradas al prompt de DeepSeek—, se exporta en PDF/DOCX corporativo y puede subirse al CRM. El módulo Campaigns encadena Opportunities → Brief → AI Assistant → Content Library → Outreach → Activación → Reports, y Outreach clasifica la intención de las respuestas con DeepSeek bajo un flujo de aprobación humana.

Componentes

Módulos Proposals, Opportunities, Content Library, Outreach, AI Assistant, Campaigns, Reports y Book Concepts, sobre una capa de seguridad con RBAC jerárquico (admin/manager/user), audit logs, rate limiting y ownership por usuario. Herramientas de análisis de contenido (plagio, detección de IA, score SEO) según la API de referencia.

Tecnología

Entorno de producción documentado: hosting en Vercel, base de datos Firestore (southamerica-west1), LLM DeepSeek (deepseek-chat) con RAG y módulos legacy en Gemini; autenticación OAuth Google y Microsoft con JWT; exportación con ReportLab y WeasyPrint; Celery/Redis previsto para v2. El diseño original contemplaba monorepo Node.js/Express + React (Vite, Tailwind, Zustand, React Query, TipTap) con PostgreSQL/Redis.

SSO Gestion de riesgo Industrial v2

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: SSO sistema de gestion de riesgos de planta industrial

Objetivo de negocio

Vigía Industrial es un Sistema Integral de Gestión de Riesgos (SSO) desarrollado para Alloxentric y entregado formalmente al cierre del primer semestre de 2026, con acta de cierre firmada y conformidad del cliente. Su propósito es que cualquier empleado de una planta industrial pueda reportar hallazgos de seguridad —actos inseguros y condiciones inseguras— de forma inmediata desde terreno, y que los supervisores y el equipo de seguridad ocupacional gestionen esos reportes hasta su cierre con acciones correctivas. El sistema pone especial énfasis en los IAP (incidentes de alto potencial): situaciones que no causaron lesiones graves pero pudieron hacerlo, y que generan alertas prioritarias.

Este proyecto es de continuidad: el equipo recibe un producto funcionando, con manuales de usuario de la aplicación móvil y del sistema web, manual de instalación y código fuente entregados, y debe desarrollar mejoras y nuevas funcionalidades sobre esa base (v2), preservando la operación y la integridad de los registros existentes.

Cómo funciona

El empleado abre la app móvil y, mediante el botón flotante "REPORTAR", crea un hallazgo en menos de 60 segundos y en un máximo de 3 clics: captura evidencia fotográfica obligatoria (opcionalmente video), selecciona la ubicación (área, línea y proceso, información que alimenta el mapa de calor del sistema), clasifica el hallazgo, marca si es IAP y agrega una descripción, que puede dictar por voz. Si no hay conexión, el reporte se guarda localmente y se envía al recuperar señal. Luego el empleado sigue el avance por un timeline de estados (Enviado → En Revisión → Acción Asignada → Cerrado), recibe feedback del supervisor y notificaciones de alertas cuando una zona de su área alcanza nivel de riesgo alto o crítico. La pantalla de inicio ofrece un dashboard personal con resumen de reportes, alertas del turno y estadísticas de efectividad; el sistema web complementa la gestión del lado de supervisores.

Componentes

Aplicación móvil del empleado (pantallas de inicio, creación de reporte, detalle y seguimiento, historial con filtros, notificaciones y perfil con estadísticas); sistema web de gestión para supervisión; motor de alertas por zonas y niveles de riesgo con mapa de calor; y la documentación entregada (manuales de usuario, de instalación y código fuente) como punto de partida de la v2.

Tecnología

La documentación disponible describe una aplicación móvil compatible con iOS 14.0+ (iPhone 8 en adelante) y Android 10.0+, con uso de cámara, dictado por voz, notificaciones push y almacenamiento local con sincronización diferida sin conexión, junto a un sistema web para supervisores. Los manuales no explicitan frameworks ni base de datos, por lo que una primera tarea del equipo será levantar y documentar el stack real a partir del código fuente y el manual de instalación entregados por Alloxentric.

Vox Ready

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: VoxReady

Objetivo de negocio

VoxReady (Alloxentric – Visum) es una plataforma web SaaS multi-cliente de autoaprendizaje que entrena a los voceros designados de una organización para gestionar la comunicación pública en situaciones de crisis. A diferencia de un curso tradicional, el usuario practica frente a la cámara respondiendo en tiempo real a un entrevistador simulado por IA y recibe una evaluación multimodal de su desempeño en cuatro áreas: expresión, tono de voz, coherencia y nivel de empatía. El diferenciador del producto es la consistencia del análisis: un "patrón maestro" definido por el equipo central (configurador maestro) fija el estándar global de buena vocería, mientras cada cliente solo personaliza el contexto de práctica (temas, mensajes clave, líneas rojas, óptica institucional y política de retención). El objetivo del MVP, en un horizonte de unos tres meses, es demostrar que este análisis entrega retroalimentación accionable y creíble.

Cómo funciona

El vocero elige un escenario configurado por el administrador de su organización, otorga el consentimiento biométrico obligatorio (UK GDPR), pasa una comprobación técnica de cámara y micrófono, y sostiene una entrevista en vivo donde un LLM formula preguntas y repreguntas mientras se transcribe su voz en streaming. La grabación se sube directamente a almacenamiento cloud mediante una URL SAS de un solo uso y, al finalizar, se encola un análisis asíncrono en tres flujos paralelos: contenido (LLM-como-juez guiado por el patrón maestro), voz/prosodia (características acústicas: ritmo, pausas, muletillas) e imagen/no verbal (landmarks faciales y de postura). Una capa de fusión integra las señales y produce un informe tipo coach con puntaje global y por área, panel de progreso, recomendaciones vinculadas a microlecciones y agenda de práctica. Los casos de baja confianza se enrutan a una cola de segunda opinión humana que calibra la IA y genera etiquetas de entrenamiento.

Componentes

API REST (154 dominios funcionales organizados por rol) para tres perfiles: vocero (escenarios, sesiones, consentimiento, informes, progreso), administrador del cliente (temas CRUD, política de retención, solicitudes de borrado) y configurador maestro (rúbrica versionada, cola de etiquetado, panel de calidad IA-humano); pipeline multimodal asíncrono; webhooks de fin de análisis; y jobs de borrado/anonimización por tenant.

Tecnología

Stack íntegramente sobre Microsoft Azure: API Management como puerta de entrada, Azure AD B2C (OAuth2/OIDC, JWT con roles por claim), App Service/Functions para API y workers, Service Bus para la cola de análisis, Blob Storage con SAS para grabaciones, Azure SQL/Cosmos DB para persistencia, Azure AI Speech, Video Indexer y Azure OpenAI para los pipelines de voz, imagen y contenido, Event Grid para webhooks y Key Vault para secretos. Gestión con Scrum adaptado: sprints de una semana, equipo de 4 (backend, frontend, base de datos, QA/documentación), 20 sprints con MVP al sprint 12.

Acredittia

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: Acredittia

Objetivo de negocio

Acredittia es una plataforma SaaS de gestión de acreditaciones de empresas contratistas, personal y equipos para faenas mineras y energéticas en Chile. Para trabajar en una faena (Los Pelambres y otras del ecosistema AMSA, entre las 9 ya integradas), cada empresa proveedora debe acreditar decenas de documentos —contratos, exámenes de altura, inducciones, F30-1, SOAP, revisiones técnicas, licencias internas de mina (EMSIPOR/LIM)— con vigencias distintas y en plataformas de mandante diferentes (SIGA, Workmate, Metacontratas, Webcontrol). Acredittia centraliza ese proceso: checklists documentales autogenerados, revisión automática de documentos con IA, alertas de vencimiento y sincronización con las plataformas del mandante. El diferenciador declarado en el roadmap no es solo la IA, sino la integración profunda por faena (procesos, formularios y contactos), difícil de replicar.

Cómo funciona

Una empresa contratista se registra y es aprobada por un administrador; luego crea contratos por faena y da de alta trabajadores y equipos, para los que el sistema instancia automáticamente los requisitos desde plantillas maestras (10 ítems de empresa, 13 de personal, 10 de equipo y 9 de EMSIPOR para conductores). Los archivos se suben directamente a Azure Blob Storage mediante SAS URLs, sin pasar por la API; un worker ejecuta OCR/LLM y valida cada documento contra la plantilla del requisito (campos clave, vigencia, RUT, firma, legibilidad), clasificando hallazgos como error, warning o info. Un cron diario recalcula estados (ok → por vencer → vencido), genera alertas y notifica por email y WhatsApp. La plataforma incluye dashboard de KPIs con tendencia histórica, matriz de cumplimiento, calendario, reportes PDF/Excel, extracción IA que pre-llena formularios desde contratos, credenciales de plataformas del mandante cifradas en JWE y auditoría completa de la actividad.

Componentes

API REST FastAPI (154 endpoints especificados; 46 implementados según el Anexo A, punto de partida real del equipo); frontend web SPA nuevo derivado del wireframe funcional `acredittia.html`; workers asíncronos para revisión IA, reportes y sincronizaciones; conectores de integración (SIGA como primer conector real); catálogos maestros (faenas, plataformas, cargos, laboratorios, talleres, proveedores GPS); y módulos de alertas, calendario, reportes y actividad.

Tecnología

Backend en Python 3.12+ con FastAPI y Pydantic v2; PostgreSQL 16 con SQLAlchemy 2.x async y migraciones Alembic (incluye RLS a nivel de fila y multi-tenancy por `company_id`); Azure Blob Storage con SAS URLs; Celery (o ARQ) + Redis para tareas asíncronas; Azure Key Vault para secretos y cifrado JWE (RSA-OAEP-256 + A256GCM) de credenciales; JWT RS256 con refresh tokens; frontend React/Next.js; notificaciones por SMTP/SendGrid y WhatsApp Business API. Plan de desarrollo de 3 meses: 6 sprints quincenales, 4 personas (BE1, BE2, FE, QA), priorización MoSCoW, GitHub Actions, pytest y Playwright.

Acredittia administracon del negocio

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: Acredittia

Objetivo de negocio

Este proyecto construye la capa de administración del negocio de Acredittia: el backoffice con el que el equipo interno opera la plataforma como empresa SaaS, complementando el producto que usan las contratistas. La documentación técnica ya especifica sus dos pilares. Primero, el módulo de administración (rol admin): aprobación o rechazo de solicitudes de registro de empresas, gestión de los catálogos globales (faenas, plataformas de mandantes, plantillas de requisitos, cargos base), métricas globales (empresas activas, pendientes, cumplimiento promedio), impersonación auditada de empresas vía header X-Company-Id y reinicio de empresas demo. Segundo, el módulo de suscripciones y pagos: modelo comercial de suscripción mensual fija sin cobro por proceso, con planes en UF, ciclo trial (30 días al aprobar la cuenta) → activa → morosa (acceso solo lectura tras 7 días de pago fallido) → cancelada, facturación y conciliación de pagos vía webhook. El roadmap comercial (julio 2026) añade contexto de negocio: base de contactos por faena (listados WRAC, p. ej. 1.142 empresas proveedoras en Los Pelambres) y proceso de ventas outbound más canal de alianzas con faenas, cuya eventual instrumentación en la plataforma queda como alcance propuesto a validar con el cliente.

Cómo funciona

El equipo de Acredittia recibe cada solicitud de registro (empresa en estado pending), la revisa y la aprueba o rechaza con notificación por email; la aprobación activa el trial de la suscripción. Desde el panel administra los catálogos que alimentan a todas las empresas —plantillas versionables por faena, cargos con sus requisitos, biblioteca de documentos de ejemplo— y monitorea métricas globales de cumplimiento. En el flujo de cobro, la empresa contrata un plan; las facturas se emiten por período y la pasarela notifica los pagos al webhook /webhooks/pagos, que actualiza facturas y reactiva suscripciones morosas. Los estados de suscripción condicionan el acceso de la empresa a la plataforma.

Componentes

Endpoints /admin (companies, stats, faenas, plataformas, plantillas de requisitos, cargos, reset-demo) y su interfaz de backoffice; módulo /planes, /suscripcion y /facturas con integración de pasarela y webhook de conciliación; modelo de datos ya definido (tablas planes, suscripciones, facturas, con estados tipados); y auditoría de toda impersonación y mutación en la tabla actividad. Cabe notar que, según el Anexo A de la especificación, estas tablas existen en el esquema SQL pero carecen aún de modelo ORM y router: es precisamente el hueco que este proyecto cierra.

Tecnología

La misma plataforma Acredittia: Python 3.12+ con FastAPI y Pydantic v2, PostgreSQL 16 (SQLAlchemy 2.x async, Alembic, multi-tenancy por company_id), workers Celery/Redis, Azure Blob Storage y Azure Key Vault, frontend React/Next.js sobre el wireframe funcional. Para pagos, la especificación plantea una base mínima con pasarela chilena (Transbank Webpay/OneClick) o Stripe, con PDFs de factura en Blob Storage y referencia de gateway por transacción.

Objetivo de negocio

AgeCare es una aplicación móvil pensada para dar "paz mental" a las familias que cuidan a un adulto mayor a distancia. Según el plan de negocio, Chile vive una transición demográfica acelerada: el 19,7% de la población (3,99 millones de personas) tiene 60 años o más y el 38,3% de los adultos mayores presenta algún grado de dependencia, con un cuidado que recae en las familias mediante herramientas fragmentadas (wearables aislados, WhatsApp, recetas en fotos). La reciente Ley 21.805 «Chile Cuida» abre además una ventana regulatoria favorable. La promesa central del producto es simple: saber, de un vistazo, que el ser querido está bien.

El modelo es freemium: el cuidado del día a día es gratis y el adulto mayor nunca paga; se monetiza a familias y proveedores con planes Dorado, Platino y Proveedor, y en fases futuras con un marketplace transaccional. El plan proyecta un TAM de US\$176M anuales en Chile y una ronda pre-seed de US\$250.000.

Cómo funciona

Es un solo producto con vistas por rol sobre datos compartidos del paciente: el familiar observa el estado de salud y recibe alertas; la cuidadora opera el día a día (check-in, tareas, confirmación de dosis, observaciones, incidentes, notas de relevo, botón SOS); el médico prescribe el plan de medicamentos; y el adulto mayor participa desde una vista radicalmente simple, accesible y de "voz primero" (chat familiar con notas de voz, galería de fotos, entretenimiento curado y check-in emocional).

El producto se organiza en cuatro pilares: salud y monitoreo (vitals del wearable con tendencias y umbrales, semáforo de bienestar diario, medicamentos y adherencia con OCR de recetas), alertas como pilar central (caída detectada, vital fuera de rango, dosis no tomada, SOS, con push y flujo crítico de acción inmediata), comunicación con IA (asistente que responde en lenguaje natural sobre el expediente y chat de coordinación con @asistente) y marketplace vitrina de cuidadoras y artículos de apoyo, con soporte multi-paciente y onboarding por invitaciones.

Componentes

App móvil multiplataforma con cuatro vistas por rol; API REST de 82 endpoints en 12 módulos (autenticación JWT, pacientes, vitals, medicamentos, bitácora, archivos, alertas, asistente IA, chat, vista elder, cuidadora, marketplace) más canal WebSocket de chat; jobs en segundo plano (generación de dosis, ventanas vencidas, vigilancia de wearable); y web de administración. El plan de desarrollo contempla 10 sprints semanales, piloto en la semana 8 y publicación en tiendas.

Tecnología

Backend en FastAPI (Python 3.12) con Pydantic v2, SQLAlchemy 2 async y PostgreSQL (Azure Flexible Server) con migraciones Alembic; Azure Blob Storage con URLs SAS, Notification Hubs/FCM-APNs para push, Azure AI Speech (TTS/STT), Azure AI Document Intelligence (OCR de recetas) y un LLM (Azure OpenAI o Claude) para el asistente. App móvil en Flutter (go_router, Riverpod/Bloc, fl_chart) para iOS y Android, WebSockets para tiempo real y CI/CD con GitHub Actions y fastlane.

Agecare modelo de generacion de musica

Tipo: Desarrollo · Modalidad: Nuevo · Carpetas: AgeCare

Objetivo de negocio

Este proyecto extiende la plataforma AgeCare —la suite de cuidado de adultos mayores documentada en su plan de negocio y funcionalidades v1— con un módulo de generación de música mediante gestos. La documentación disponible de AgeCare no detalla este módulo, por lo que lo que sigue se plantea como alcance propuesto, coherente con la visión del producto: la vista del adulto mayor prioriza el acompañamiento y el entretenimiento por sobre los datos, con principios de diseño de accesibilidad radical (mosaicos grandes, voz primero, mínimo esfuerzo). Hoy esa vista incluye chat familiar, fotos y contenido curado (chistes y noticias); un módulo musical gestual agregaría una experiencia activa y lúdica.

El objetivo propuesto es doble: entretenimiento con valor terapéutico —la creación musical mediante movimiento se asocia a estimulación cognitiva y motora en adultos mayores— y participación: que el adulto mayor pase de consumir contenido a crear algo propio que pueda compartir con su familia, reforzando el vínculo que es el corazón de AgeCare y aportando diferenciación a los planes premium de la plataforma.

Cómo funciona

Se propone que el adulto mayor abra el módulo desde un mosaico grande de su vista accesible y, con gestos simples de manos o brazos frente a la cámara del dispositivo (o sobre la pantalla táctil), genere música en tiempo real: la altura, la intensidad y el instrumento responderían a la posición y velocidad del movimiento, sin partituras ni conocimientos previos. El sistema contemplaría escalas y acompañamientos predefinidos que garanticen un resultado siempre armónico ("no hay notas equivocadas"), sesiones guiadas de ejercicio suave con música como retroalimentación, y la posibilidad de grabar una pieza y enviarla como mensaje a la galería o al chat familiar, siguiendo el diseño "de dos caras" de la plataforma. Los datos de uso (frecuencia, amplitud de movimiento) podrían alimentar la bitácora del expediente como observaciones de actividad.

Componentes

Como alcance propuesto: (1) módulo de captura y reconocimiento de gestos sobre cámara o pantalla táctil; (2) motor de mapeo gesto→parámetro musical y síntesis/secuenciación en tiempo real con baja latencia; (3) interfaz accesible integrada a la vista del adulto mayor (tipografía grande, alto contraste, instrucciones por voz); (4) grabación y compartición de piezas vía los servicios existentes de archivos y chat de AgeCare; y (5) endpoints de integración con el backend para persistir sesiones y métricas.

Tecnología

Se propone integrarse al stack documentado de AgeCare: cliente Flutter (iOS/Android) y backend FastAPI + PostgreSQL en Azure con Blob Storage para los audios. Para el reconocimiento de gestos, modelos de visión sobre el dispositivo (p. ej. MediaPipe/ML Kit para tracking de manos y pose); para la generación sonora, un motor de audio en tiempo real (síntesis o sampler embebido) y, opcionalmente, modelos generativos de música. El equipo deberá evaluar latencia, rendimiento en dispositivos de gama media y usabilidad con usuarios mayores reales.

Agecare (administración)

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: AgeCare

Objetivo de negocio

Este proyecto aborda la web de administración de AgeCare, la plataforma de cuidado de adultos mayores. Aunque la app móvil es la cara visible del producto, el plan de desarrollo v1 define el panel de administración como un epic obligatorio (EP-13, prioridad Must, presente en los 10 sprints): sin él no hay soporte a usuarios, ni curación del contenido de entretenimiento, ni moderación del marketplace, ni visibilidad operacional del negocio. Para una startup freemium que proyecta decenas de miles de usuarios gratuitos y de pago, el panel es la herramienta con la que el equipo interno opera, da soporte y toma decisiones, además de sostener requisitos de publicación en tiendas (páginas legales y privacidad de datos de salud).

El objetivo del proyecto es construir ese portal interno de punta a punta, sobre la misma API del backend AgeCare, complementando con funcionalidades propuestas de gestión comercial (residencias, planes y suscripciones) donde la documentación es más general.

Cómo funciona

Según los tickets del plan de desarrollo, usuarios internos con roles diferenciados (admin, soporte, editor de contenido) acceden a una SPA con login JWT y rutas protegidas por rol, donde toda acción de escritura queda registrada en una bitácora de auditoría. El equipo de soporte busca usuarios y pacientes, revisa membresías, desactiva cuentas y reenvía invitaciones; el editor cura el feed de chistes y noticias del adulto mayor y administra el catálogo de productos del marketplace; operaciones monitorea alertas, configura umbrales por defecto por vital y parámetros del semáforo, gestiona suscripciones premium, y consulta dashboards de adopción (usuarios por rol, pacientes activos, invitaciones) y del piloto. Incluye además herramientas de datos demo para QA y ventas, auditoría del asistente IA, publicación de ofertas de trabajo, moderación de reseñas y páginas legales públicas.

Como alcance propuesto adicional, coherente con el plan de negocio, el panel puede extenderse a la administración comercial de la plataforma: altas de cuidadoras y agencias proveedoras, gestión de familias y de los planes Dorado/Platino/Proveedor, y reportes de conversión freemium.

Componentes

Frontend SPA de administración con layout, navegación por módulos (usuarios, contenido, catálogos, moderación, alertas, configuración, dashboards, legales); endpoints admin dedicados en el backend con modelo de permisos internos y auditoría; y despliegue continuo en la nube. Se coordina con la API principal de AgeCare (82 endpoints documentados) como fuente de datos.

Tecnología

Stack documentado en el plan de desarrollo: SPA en React con Vite y TypeScript, desplegada en Azure Static Web Apps con deploy automático, autenticación JWT contra la API FastAPI (Python) y base de datos PostgreSQL en Azure; Application Insights para observabilidad y Key Vault para secretos. El proyecto ejercita diseño de paneles internos, control de acceso por roles, auditoría y visualización de métricas.

TryAI Gap

Tipo: Desarrollo · · Modalidad: Nuevo · Carpeta: TryAIGap

Objetivo de negocio

TryAIGap ("Bridge the AI Gap") es una plataforma multitenant de consultoría online que guía a organizaciones a través de un diagnóstico de madurez en inteligencia artificial. Se apoya en una metodología propia de siete módulos que ataca la causa por la que entre el 70% y el 85% de los proyectos de IA empresariales no entregan el valor esperado: el método, no la tecnología. El producto digitaliza la puerta de entrada a esa consultoría con un modelo comercial freemium: cualquier ejecutivo con correo corporativo inicia gratis el diagnóstico de madurez (Módulo 1, 5 dimensiones), ve un resultado parcial y desbloquea mediante pago el diagnóstico completo por áreas funcionales, con revisión opcional por consultores humanos. La plataforma opera en cuatro idiomas (ES/EN/DE/PT) y aísla los datos por organización (multi-tenant), con roles de cliente, consultor y administrador.

Cómo funciona

El usuario entra por un lead gate que valida el correo corporativo (rechaza dominios gratuitos) y crea su organización en plan free; la autenticación es sin contraseña, por magic link con tokens JWT. Responde el diagnóstico de madurez y luego los cuestionarios por área funcional (7 áreas de 16 preguntas; 132 preguntas en total con traducciones), con escala 1–5, opción "no sé", carga de evidencias documentales y delegación de preguntas a colegas mediante token de un solo uso. Al alcanzar el límite de la vista gratuita aparece el paywall: un estimador cotiza en vivo la configuración elegida (áreas, revisiones, sesiones, informe final, código de distribuidor con descuento) y el pago con Stripe activa el plan Pro. El motor de resultados calcula radar de madurez, heatmap de áreas x vectores de valor, matriz Pain x Readiness y priorización de iniciativas (Quick Win / Invertir / Aún no), con informe ejecutivo en PDF. Opcionalmente, un consultor revisa el engagement por capítulos desde su propio panel y el cliente lo evalúa.

Componentes

Backend API REST (auth por magic link, leads, organizaciones, cuestionarios, áreas, documentos, equipo/invitaciones/delegación, estimador, pagos, resultados, revisión humana y panel de consultor); aplicación web SPA con design system propio, tema claro/oscuro e i18n; app móvil (diferida en el repo actual, solo rol cliente); y kits Excel en español e inglés como fuente sembrada de los cuestionarios.

Tecnología

Monorepo con backend en FastAPI (Python 3.11+), SQLAlchemy 2.x async y Alembic sobre PostgreSQL 15+ en producción (SQLite en desarrollo), validación con Pydantic v2 y contrato OpenAPI con generación de tipos compartidos; frontend web en React + TypeScript + Vite + Tailwind + shadcn/ui, i18n con react-i18next e informe PDF generado en cliente con @react-pdf/renderer; pagos con Stripe (simulado en dev); móvil en Flutter diferido según el README (el plan original contemplaba React Native). Plan de desarrollo de 12 sprints semanales con equipo de 4 personas (~325 story points), MVP web al Sprint 6 y release al Sprint 12.

WellQ App Exámenes médicos

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: carpeta vacía

Objetivo de negocio

WellQ es una plataforma de salud digital desarrollada para una clínica de fisioterapia del Reino Unido, cuyo núcleo es una aplicación móvil de pacientes que hace seguimiento del progreso durante los planes de rehabilitación, integrando datos de wearables y documentación clínica generada en consulta. En ese ecosistema, los exámenes médicos (analíticas, informes de imagen, resultados de laboratorio, informes de derivación) circulan hoy fuera de la plataforma, por correo o en papel, lo que fragmenta la historia del paciente y complica el trabajo del fisioterapeuta.

Este proyecto, según la descripción de la planilla, aborda el **uploading de exámenes médicos multipaciente y multitenant** en los tres niveles de la plataforma: app móvil, backend y frontend clínico. El objetivo de negocio es que cada clínica (tenant) gestione de forma segura y aislada los documentos de todos sus pacientes, y que el clínico los consulte junto al resto de datos de la sesión. La carpeta del proyecto está vacía, por lo que el detalle funcional que sigue debe leerse como alcance propuesto sobre la base documentada de la plataforma WellQ.

Cómo funciona

Como alcance propuesto: el paciente (o el personal de la clínica) sube un examen desde la app o desde el panel web, indicando tipo de documento, fecha y profesional asociado. El backend valida el archivo (tipo MIME, tamaño), lo almacena en un bucket de objetos con una clave con alcance de tenant (por ejemplo ``clinics/{clinicId}/patients/{patientId}/exams/{uuid}``) y registra los metadatos en base de datos. Toda consulta queda restringida por el middleware de alcance multitenant, siguiendo el patrón ya documentado en la Clinical API de WellQ: un clínico de la Clínica A nunca puede acceder a documentos de la Clínica B. El fisioterapeuta visualiza los exámenes del paciente en una línea de tiempo, junto a las sesiones y notas clínicas, con control de versiones y registro de auditoría de cada acceso (requisito de cumplimiento UK GDPR).

Componentes

- Módulo de subida y visualización en la app móvil de pacientes (propuesto).
- Endpoints REST de carga multipart, listado paginado, descarga y borrado lógico de exámenes, con aislamiento por ``clinic_id``.
- Almacenamiento de objetos (S3/MinIO) y modelo de metadatos en base de datos relacional.
- Panel web/front clínico para revisión de documentos y auditoría de accesos (propuesto).

Tecnología

Stack propuesto, alineado con el documentado en el ecosistema WellQ: backend Node.js 20 con TypeScript y Express/Fastify, PostgreSQL 16 con migraciones versionadas y borrado lógico, MinIO/S3 para archivos, autenticación JWT con middleware de tenant, validación con Zod, Docker Compose para desarrollo local, tests con Jest/Supertest y documentación OpenAPI/Swagger; app móvil en Flutter/Dart, consistente con el cliente WellQ existente.

WellQ App GLP1 Skin

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: carpeta vacía

Objetivo de negocio

WellQ opera una plataforma de salud digital para una clínica del Reino Unido, centrada en fisioterapia y bienestar: una app móvil de pacientes que integra datos de wearables (frecuencia cardíaca, pasos, sueño, entrenamientos) a través de una capa de APIs unificada, más un backend clínico multitenant. Sobre esa base, la empresa quiere atender a un segmento en fuerte crecimiento: los pacientes en tratamiento con **inhibidores/agonistas GLP-1** (medicamentos para control de peso y metabólico como semaglutida o tirzepatida), cuyo acompañamiento requiere preservar masa muscular, monitorizar actividad física y hábitos, y dar seguimiento a la adherencia.

Según la planilla, el proyecto consiste en el **desarrollo de una versión de app reducida ("skin") para pacientes usuarios de GLP-1**: una variante de la app WellQ con un subconjunto de funcionalidades y una experiencia adaptada a este perfil. La carpeta del proyecto está vacía, por lo que la definición funcional detallada que sigue constituye alcance propuesto, construido sobre la arquitectura documentada de WellQ.

Cómo funciona

Como alcance propuesto: la app GLP-1 reutiliza el código base de la app WellQ (misma autenticación HMAC/JWT contra el backend, mismos servicios de datos de wearables) pero con una configuración de "skin" que activa solo las pantallas relevantes: registro de peso y composición corporal, actividad diaria (pasos, entrenamientos de fuerza para preservar masa muscular), sueño, y un diario simple de dosis y efectos. Los datos de wearables llegan por los mismos endpoints ya documentados de la plataforma (timeseries de `heartrate` y `steps`, workouts, sleeps, métricas como `weight`, `body\_fat` y `body\_mass\_index` disponibles en la API de consulta). El equipo clínico ve la evolución del paciente en el backend existente. El reto de ingeniería es el mecanismo de variante: feature flags o build flavors que permitan compilar la app reducida sin bifurcar el repositorio principal, manteniendo la modalidad de continuidad sobre el producto vivo.

Componentes

- App móvil "skin" GLP-1: onboarding específico, dashboard reducido (peso, actividad, sueño), diario de medicación (propuesto).
- Sistema de configuración de variantes (flavors/feature flags) sobre el código base de la app WellQ.
- Ajustes de backend: perfil de paciente GLP-1, métricas objetivo y alertas de inactividad o pérdida acelerada de peso (propuesto).

Tecnología

App móvil en Flutter/Dart, consistente con el cliente WellQ documentado (servicios como `spike\_auth\_service.dart` y `spike\_timeseries\_service.dart`), con build flavors para la variante GLP-1; backend existente Node.js/TypeScript con PostgreSQL, autenticación JWT y APIs de datos de salud con métricas de peso, composición corporal, actividad y sueño. Todo lo no cubierto por la documentación disponible (diario de medicación, alertas) se plantea como stack propuesto coherente con la plataforma.

WellQ API

Tipo: Desarrollo · Modalidad: Continuidad · Carpeta: Proyecto APIs WellQ

Objetivo de negocio

WellQ es la aplicación móvil de fisioterapia de una clínica del Reino Unido cuyo valor diferencial es el seguimiento objetivo de la rehabilitación mediante datos de wearables. Hoy la app consume la API de Spike (spikeapi.com), una capa unificada de integración con más de 15 proveedores (Apple HealthKit, Garmin, Fitbit, Whoop, Oura, Polar, Ultrahuman, entre otros). Para reducir la dependencia de ese intermediario, la documentación del proyecto plantea construir un backend propio que replique los endpoints de Spike que WellQ consume, incorporando de forma nativa nuevos proveedores. Según la planilla, este equipo desarrolla el **soporte de Ultrahuman, Fitbit y hltid** para la app; la API Reference del proyecto documenta además Whoop, Garmin y Oura como dispositivos a integrar nativamente.

El objetivo académico y de negocio es que, al final del semestre, la app WellQ pueda apuntar al servidor del equipo en lugar del oficial, con comportamiento equivalente, y con datos reales de los proveedores asignados.

Cómo funciona

La autenticación está completamente documentada: el backend genera una firma HMAC-SHA256 del ``application_user_id`` con una clave secreta compartida (implementada hoy en ``spike_auth_service.dart`` de la app Flutter), la intercambia en ``POST /v3/auth/hmac`` por un token JWT de 24 horas, y todo endpoint posterior exige ``Authorization: Bearer <token>``. Sobre esa base, la app consume nueve endpoints de consulta: ``GET /v3/queries/timeseries`` (el más usado, series de ``heartrate`` y ``steps`` con arrays paralelos de offsets, durations y values), ``workouts`` (historial de entrenamientos con tipo, duración, calorías y FC promedio), ``sleeps`` (fases profundo/ligero/REM/desperto, con fechas en zona horaria local y caché de 15 minutos en la app), ``records`` y estadísticas por intervalo y diarias. La gestión de proveedores usa el flujo OAuth documentado: ``GET /v3/providers/{slug}/integration/init_url`` devuelve la URL de consentimiento, ``DELETE .../integration`` desconecta el dispositivo y ``GET /v3/userinfo`` lista los proveedores integrados (Ultrahuman requiere además ``provider_user_id``). El plan de hitos documentado divide el semestre en autenticación, datos de salud I y II, gestión de proveedores y demo final integrada.

Componentes

Servidor de autenticación HMAC/JWT con middleware de validación; endpoints de consulta (timeseries, workouts, sleeps, records, statistics) sobre un modelo de datos unificado; módulo de integración nativa con las APIs de Fitbit y Ultrahuman (OAuth, normalización de métricas); gestión de proveedores y userinfo; documentación OpenAPI y colección de pruebas.

Tecnología

Stack sugerido por la documentación real: backend en Node.js (Express/Fastify), Python (FastAPI/Django) o Go; PostgreSQL o MongoDB para registros de salud; HMAC-SHA256 y JWT (jsonwebtoken/PyJWT); pruebas con Postman/Insomnia y documentación Swagger/OpenAPI. Cliente móvil existente en Flutter/Dart. Métricas mínimas a soportar: heartrate, steps, hrv_rmssd, spo2, breathing_rate y calorías activas.

WellQ Generic App for manufacturers

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: sin carpeta

Objetivo de negocio

WellQ ha construido, para su clínica de fisioterapia en el Reino Unido, una app de pacientes que integra datos de más de 15 proveedores de wearables (Apple HealthKit, Garmin, Fitbit, Whoop, Oura, Ultrahuman, etc.) mediante una capa de APIs unificada, y un backend clínico multitenant. Esa infraestructura tiene valor más allá de la clínica: muchos **fabricantes de hardware** de salud y fitness (anillos, bandas, básculas, sensores) necesitan ofrecer una app de acompañamiento a sus dispositivos, pero no quieren desarrollar software desde cero.

Según la planilla, el proyecto consiste en el **desarrollo de una API y un generador de app whitelabel para fabricantes de hardware**: convertir la app WellQ en un producto configurable que cualquier fabricante pueda lanzar con su marca, conectada a sus dispositivos. No existe carpeta de documentación para este proyecto, por lo que todo el alcance funcional descrito es propuesto, apoyado en la arquitectura real ya documentada de la plataforma WellQ (autenticación HMAC/JWT, modelo unificado de métricas, multitenancy por organización con configuración de branding en base de datos).

Cómo funciona

Como alcance propuesto: el fabricante se registra a través de una API de administración, define su configuración whitelabel (nombre, logotipo, paleta de colores, pantallas habilitadas, proveedores/dispositivos soportados, textos legales) y obtiene credenciales de tenant. Un **generador de apps** toma esa configuración y produce una build móvil personalizada a partir del código base de WellQ: mismo motor de datos de salud (series de tiempo, entrenamientos, sueño), pero con la identidad visual y el catálogo de dispositivos del fabricante. La API expone además los datos normalizados de los usuarios finales del fabricante, con aislamiento estricto por tenant, siguiendo el patrón de scope middleware documentado en la plataforma. El flujo de conexión de dispositivos reutiliza el esquema OAuth de integración de proveedores ya documentado (URL de consentimiento, desconexión, listado de integraciones).

Componentes

- API de administración de tenants/fabricantes: registro, credenciales, configuración de branding (JSONB) y catálogo de dispositivos (propuesto).
- Generador whitelabel: pipeline que compila la app móvil con la configuración del fabricante (iconos, colores, flavors) (propuesto).
- API de datos de salud multitenant reutilizando el modelo unificado de métricas de WellQ.
- Portal o CLI de autoservicio para que el fabricante previsualice y publique su app (propuesto).

Tecnología

Stack propuesto, coherente con el ecosistema documentado: backend Node.js 20/TypeScript (Express/Fastify) con PostgreSQL 16 (configuración de branding en JSONB, multitenancy con scope por tenant), autenticación HMAC-SHA256 + JWT como la plataforma actual, Docker y CI/CD con GitHub Actions para el pipeline de generación de builds; app base en Flutter/Dart con build flavors y theming dinámico para el whitelabel; documentación OpenAPI/Swagger.

WellQ AI driven Exercise Generator

Tipo: Desarrollo · Modalidad: Nuevo · Carpeta: sin carpeta

Objetivo de negocio

En la plataforma WellQ —salud digital para una clínica de fisioterapia del Reino Unido— el plan de ejercicios es el corazón del tratamiento: los documentos clínicos reales del ecosistema muestran que las cartas y notas generadas por la plataforma incluyen progresiones de ejercicios con bandas de resistencia, series y repeticiones específicas para cada paciente. Sin embargo, producir el material de apoyo de esos ejercicios (fichas técnicas y videos demostrativos) es hoy un trabajo manual costoso para el equipo clínico.

Según la planilla, este proyecto nuevo aborda el **desarrollo de un generador de especificaciones y videos de ejercicios**: un sistema asistido por IA que produce, a partir de una petición clínica, la especificación estructurada de un ejercicio terapéutico (nombre, objetivo, músculos implicados, material, series/repeticiones, precauciones, progresiones y regresiones) y un video demostrativo asociado, alimentando la biblioteca de ejercicios de la app de pacientes. No existe carpeta de documentación, por lo que todo el alcance descrito es propuesto, apoyado en el contexto real de la plataforma.

Cómo funciona

Como alcance propuesto: el fisioterapeuta describe la necesidad ("fortalecimiento de manguito rotador, fase intermedia, con banda elástica") y el generador, usando un modelo de lenguaje (siguiendo el patrón ya documentado en la plataforma, que usa la API de Anthropic Claude para generar notas clínicas SOAP y cartas), produce una especificación en JSON validada contra un esquema clínico, con ejemplos curados como few-shot y revisión obligatoria del profesional antes de publicar. En paralelo, un pipeline genera el video demostrativo mediante modelos generativos de video/avatares o composición programática (animación de esqueleto/render 3D) con locución sintetizada. Cada ejercicio aprobado se versiona y se publica en la biblioteca, desde donde se asigna a los planes de rehabilitación que el paciente sigue en la app, cerrando el ciclo con los datos de adherencia y de wearables que la plataforma ya recoge.

Componentes

- Servicio de generación de especificaciones con LLM: prompts con esquema JSON, validación y flujo de revisión/aprobación clínica (propuesto).
- Pipeline de generación de video demostrativo con cola de trabajos asíncrona (propuesto).
- Biblioteca de ejercicios: API CRUD versionada, almacenamiento de videos en objeto (S3), metadatos y etiquetado por patología/fase (propuesto).
- Integración con la app de pacientes para reproducir videos dentro del plan (propuesto).

Tecnología

Stack propuesto, alineado con el ecosistema documentado: backend Node.js 20/TypeScript (Express/Fastify), PostgreSQL 16, Redis con BullMQ para los trabajos asíncronos de generación, S3/MinIO para videos, API de Anthropic Claude para las especificaciones (patrón ya usado en la Clinical API de WellQ), un servicio de generación o composición de video (modelos generativos, render 3D o plantillas animadas) y front en Flutter/Dart para el consumo en la app; documentación OpenAPI y despliegue con Docker y GitHub Actions.

Chat de Soporte Generico Alloxentric

1. Introducción

Alloxentric es una empresa especializada en la automatización de comunicaciones y atención al cliente. Cada empresa que utiliza sus servicios —y, en general, cualquier producto SaaS— necesita atender consultas y problemas de sus usuarios finales; hacerlo con soporte humano dedicado por producto es costoso y no escala. Este proyecto aborda ese problema mediante una plataforma de soporte conversacional genérica y reutilizable, que cualquier empresa cliente de Alloxentric pueda configurar para su propia aplicación sin necesidad de desarrollo a medida.

El proyecto consiste en construir un módulo de administración de un chatbot de soporte disponible en dos canales, web y WhatsApp, que responde a los usuarios finales a partir del conocimiento propio de cada empresa. Cada cliente carga en el módulo de administración su documentación, sus preguntas frecuentes (FAQ) y la URL de su sitio web. Con esa información la plataforma construye una base de conocimiento y genera un enlace hacia un chat de soporte que utiliza un esquema RAG (Retrieval-Augmented Generation) conectado al modelo de lenguaje DeepSeek, con una API key independiente para cada cliente.

El chatbot de cada empresa no es un bot único: se compone de dos agentes especializados —uno de soporte técnico y uno de soporte comercial— con instrucciones, tono y fuentes de conocimiento propias. Un clasificador de intención decide cuál responde cada consulta, y cuando el bot no puede resolver, la conversación se deriva a un agente humano del tipo correspondiente, en vivo si hay alguien disponible o como ticket si no lo hay.

La plataforma nace multi-tenant desde su diseño: una sola implementación sirve a múltiples empresas, cada una con su base de conocimiento, su API key, sus agentes humanos y su enlace de chat, con aislamiento total de la información entre clientes. La interfaz de administración está disponible en español, inglés y portugués, con modo claro y oscuro, y el bot responde en el idioma en que escribe el usuario. El proyecto partirá con cuatro clientes iniciales: WellQ, Alloxentric, Directa y Dani27001.

Este documento acompaña al wireframe funcional (Wireframe_Chatbot_Soporte_v5.html), que muestra de forma navegable las tres interfaces de la plataforma y cuyas definiciones se recogen aquí.

2. Objetivos

2.1 Objetivo general

Desarrollar una plataforma multi-tenant de soporte conversacional, compuesta por un módulo de administración, un chat de soporte web y WhatsApp con agentes especializados (técnico y comercial) basados en RAG y DeepSeek, y una consola para agentes humanos que reciben las conversaciones derivadas, de modo que cualquier empresa cliente de Alloxentric ofrezca a sus usuarios una primera línea de atención automatizada con respaldo humano.

2.2 Objetivos específicos

- Construir un **módulo de administración** donde cada empresa cliente gestione su cuenta, sus documentos, sus preguntas frecuentes, la URL de su sitio, sus agentes del bot y sus agentes humanos.
- Implementar un **pipeline de ingesta de conocimiento** que procese los documentos subidos, las FAQ y el contenido del sitio web (escaneo/crawling) para construir una base de conocimiento vectorizada por cliente, donde cada documento pertenece al tipo de chat que lo usará (técnico o comercial).
- Desarrollar el **motor conversacional RAG** con dos agentes especializados por empresa (soporte técnico y soporte comercial) y un clasificador de intención que dirige cada consulta al agente correcto, generando respuestas mediante DeepSeek con la API key propia de cada cliente.
- Generar automáticamente un **enlace de chat de soporte** por cliente y habilitar el **canal WhatsApp** sobre el mismo motor conversacional.

- Implementar la **derivación a agentes humanos**: chat en vivo cuando hay un agente disponible del tipo correspondiente y creación de ticket cuando no lo hay, con una consola de agente que incluye cola de conversaciones, transcripción con contexto y respuestas sugeridas por el motor RAG.
- Mantener vivo el conocimiento mediante **sugerencias de FAQ generadas por IA** a partir de las conversaciones reales, y permitir la exportación de las FAQ a PDF, JSON y Word.
- Garantizar el **aislamiento multi-tenant** y ofrecer al administrador de la plataforma visibilidad consolidada: actividad, ficha de cada empresa y consumo de tokens por período.
- Ofrecer una interfaz **multiidioma (español, inglés y portugués)** con modo claro y oscuro, y un bot que responde en el idioma del usuario.
- Poner en producción la plataforma con los **cuatro clientes iniciales** (WellQ, Alloxentric, Directa y Dani27001) y validar su operación real.

3. Alcance

3.1 Dentro del alcance

- Módulo de administración web multi-tenant con dos niveles: administrador de plataforma (Alloxentric) y administrador de empresa, en español, inglés y portugués y con modo claro/oscuro.
- Gestión de documentos del cliente: ver, subir y editar documentos (PDF, Word, texto, Markdown), cada uno asignado al chat técnico, al comercial o a ambos; escaneo del sitio web del cliente; edición de FAQ con sugerencias de IA y exportación a PDF, JSON y Word.
- Dos agentes del bot configurables por empresa (técnico y comercial): nombre, temas, instrucciones (prompt), fuentes de conocimiento y destino de derivación humana; el comercial puede activarse o desactivarse.
- Gestión de agentes humanos por empresa: crear, modificar y eliminar, indicando qué tipo de agente del bot les deriva (técnico, comercial o ambos) y el canal de derivación (chat web en esta versión). El administrador de plataforma los ve todos en modo lectura.
- Chat de soporte web con enlace único por cliente y widget embebible; canal WhatsApp (bot) mediante la API oficial de WhatsApp Business.
- Derivación a humano: chat en vivo con consola de agente (cola, transcripción con motivo y contexto, respuestas sugeridas por RAG) y tickets cuando no hay agentes disponibles.
- Conexión con DeepSeek mediante API key configurable e independiente por cliente, almacenada cifrada.
- Dashboards: por empresa, con indicadores clickeables que muestran el detalle (conversaciones, resueltas por el bot, tickets pendientes derivados y derivaciones); global de plataforma, con ficha de cada empresa (razón social, RUT/ID fiscal, país, idioma), bots por canal en uso y consumo de tokens con selector de período (hoy, últimos 7 días, últimos 30 días, mes anterior, últimos 3 meses y año).

3.2 Fuera del alcance (primera versión)

- Derivación a agentes humanos por WhatsApp (en esta versión los humanos atienden solo por chat web; el bot sí opera en WhatsApp).
- Canales adicionales como telefonía/voz, Telegram o redes sociales.
- Facturación automática a los clientes por uso de la plataforma.

4. Cómo funciona

El flujo de la plataforma tiene tres vistas: la de la empresa cliente, que configura su chatbot; la del usuario final, que lo utiliza; y la del agente humano, que atiende lo que el bot deriva.

4.1 Flujo de la empresa cliente (administración)

- **1. Alta del cliente:** el administrador de plataforma crea la empresa (razón social, RUT/ID fiscal, país, idioma) con sus usuarios administradores y su API key de DeepSeek.
- **2. Carga de conocimiento:** el cliente sube sus documentos indicando a qué chat pertenecen (técnico, comercial o ambos), registra sus FAQ y la URL de su sitio, y puede lanzar el escaneo del sitio cuando lo necesite.
- **3. Configuración de agentes:** ajusta los dos agentes del bot (técnico y comercial) y crea a sus agentes humanos, indicando qué tipo de derivación recibe cada uno.
- **4. Publicación:** la plataforma genera el enlace único del chat, el código del widget embebible y habilita el número de WhatsApp asociado.
- **5. Mejora continua:** el módulo sugiere nuevas FAQ a partir de las preguntas reales más frecuentes que aún no están respondidas; el cliente las revisa y añade con un clic, y el documento de FAQ crece y se reindexa.

4.2 Flujo del usuario final (soporte)

- **1.** El usuario abre el enlace del chat web o escribe al WhatsApp de soporte de la empresa y plantea su consulta en lenguaje natural, en español, inglés o portugués.
- **2.** El clasificador de intención determina si la consulta es técnica o comercial y la entrega al agente del bot correspondiente.
- **3.** El agente recupera los pasajes relevantes de sus propias fuentes (documentos de su tipo, FAQ, sitio web) y DeepSeek genera la respuesta con la API key del cliente.
- **4.** Si el bot no resuelve tras dos intentos o el usuario lo pide, la conversación se deriva: a chat en vivo con un agente humano del tipo correspondiente si hay alguien disponible, o a un ticket si no lo hay (por ejemplo, fuera de horario).
- **5.** El usuario recibe seguimiento de su caso por el mismo canal y puede valorar la atención al cierre (👍 / 🙋).

4.3 Flujo del agente humano (consola de agente)

- **1.** El agente ve su bandeja con dos colas: conversaciones en vivo esperando atención y tickets pendientes, cada una etiquetada con la empresa, el canal, el tipo (técnico/comercial) y la prioridad.
- **2.** Al tomar una conversación accede a la transcripción completa usuario-bot, el motivo de la derivación, los datos del usuario y el sentimiento detectado.
- **3.** El motor RAG le sugiere respuestas basadas en la documentación del cliente, que puede editar antes de enviar; también puede devolver la conversación al bot o cerrarla como resuelta.

5. Arquitectura de la solución

La solución se estructura en los siguientes componentes sobre una base de datos y una capa de servicios comunes. Todos son multi-tenant: cada operación se ejecuta siempre en el contexto de una empresa cliente.

Componente	Descripción
Módulo de administración	Aplicación web (ES/EN/PT, tema claro/oscuro) para gestionar tenants, usuarios, API keys, documentos por tipo de chat, FAQ con sugerencias de IA y exportación, escaneo del sitio, agentes del bot, agentes humanos, canales y apariencia. Genera el enlace de chat por cliente.
Pipeline de ingesta	Procesa documentos (PDF, DOCX, TXT, MD), FAQ y el sitio web (escáner/crawler). Limpia, trocea y vectoriza el contenido y lo indexa en la base vectorial del tenant, etiquetado por tipo de chat (técnico/comercial).
Motor conversacional	Clasificador de intención + dos agentes RAG especializados por empresa (técnico y comercial), cada uno con sus instrucciones y sus fuentes. Recupera pasajes relevantes y arma el prompt para el modelo de lenguaje.
Integración DeepSeek	Cliente de la API de DeepSeek con una API key por tenant (cifrada), control de consumo de tokens por período, reintentos y manejo de errores, tras una capa de abstracción de proveedor LLM.
Canales de chat	Chat web (enlace único por cliente, widget embebible) y canal WhatsApp mediante la API de WhatsApp Business, ambos sobre el mismo motor conversacional. El bot responde en el idioma del usuario.
Derivación y consola de agente	Enrutador de derivaciones (en vivo o ticket, según disponibilidad y tipo) y consola web para agentes humanos: colas, transcripción con contexto, respuestas sugeridas por RAG y gestión de tickets.
Métricas y reporting	Dashboards por empresa (KPIs con detalle clickeable y tickets pendientes) y de plataforma (actividad consolidada, ficha de empresa y consumo de tokens por período), con exportaciones.

5.1 Stack tecnológico propuesto

Capa	Tecnología propuesta
Backend / API	Python con FastAPI (API REST, autenticación JWT y control de acceso por rol y por tenant)
Base de datos	PostgreSQL para datos de negocio y pgvector (o una base vectorial dedicada como Qdrant) para los embeddings
Ingesta y procesamiento	Workers asíncronos (Celery/Redis) para procesar documentos y escanear sitios sin bloquear la aplicación
Modelo de lenguaje	DeepSeek vía API (chat/completions), con API key independiente por cliente; embeddings con modelo open source o servicio equivalente
Frontend	SPA moderna (React o Vue 3 con TypeScript) con i18n (ES/EN/PT) y theming claro/oscuro, para el módulo de administración, la consola de agente y el chat web
WhatsApp	WhatsApp Business Cloud API (webhooks de recepción y envío de mensajes)
Tiempo real	WebSockets para el chat en vivo agente-usuario y las notificaciones de la consola

Proyecto Paralegal (Justiniano)

1. ¿Qué es Justiniano?

Justiniano es una plataforma **SaaS (software como servicio)** que ofrece orientación legal a micro, pequeñas y medianas empresas mediante inteligencia artificial, comenzando por la jurisdicción de Chile. El producto combina tres capacidades: agentes de IA especializados por área legal que responden consultas con base normativa local, la generación automática de unos 70 formatos de documentos legales (contratos, finiquitos, anexos, etc.) y — su elemento diferenciador — una red de abogados revisores humanos que certifican los documentos con un compromiso de plazo (SLA) de 8 o 48 horas hábiles.

Como caso de estudio, Justiniano es interesante porque reúne en un solo proyecto las tres dimensiones que un gestor de proyectos debe integrar: un **modelo de negocio** (freemium con planes de suscripción), una **operación de servicio** (la red de abogados con plazos comprometidos) y un **desarrollo de software** planificado en 12 semanas con un equipo de 4 personas.

2. Objetivo de negocio

2.1 El problema que resuelve

Las mipymes concentran el 99% de las empresas de América Latina y el 61% del empleo formal (CEPAL), pero la gran mayoría opera **sin asesoría legal continua**: un abogado corporativo es un costo fijo inalcanzable y la consulta puntual con un estudio jurídico es cara, lenta e intimidante. El resultado son contratos defectuosos, finiquitos mal calculados, multas evitables y conflictos que terminan judicializados por falta de prevención.

Los sustitutos que la pyme usa hoy son imperfectos: plantillas genéricas de internet (sin jurisdicción ni actualización normativa), el contador (que no es abogado) o un chatbot de IA de propósito general, que no conoce la normativa local en profundidad y, sobre todo, **no responde por el resultado**. La oportunidad de negocio es ocupar el punto medio inexistente: orientación legal instantánea y económica, con opción de certificación humana bajo demanda.

2.2 El modelo de negocio: freemium

El objetivo comercial es captar masivamente empresas con un **plan Gratuito** (1 documento y 30 preguntas al mes) y convertir un porcentaje de ellas en clientes de pago. Existen tres planes de suscripción — Súper Básico, Intermedio y Avanzado, con precios de 1,5 a 10 UF mensuales — más un plan Corporativo a medida y la venta transaccional de créditos de revisión. Denominar los precios en UF indexa el ingreso a la inflación chilena sin necesidad de subir precios.

La lógica económica que sostiene el modelo es clave para entender el proyecto: responder una pregunta con IA cuesta ~US\$0,01 y la revisión humana de un documento cuesta ~US\$10, mientras que un crédito de revisión se vende a ~US\$33 y el cliente de pago promedio aporta ~US\$153 mensuales. El costo caro (el abogado) solo se incurre cuando hay una venta asociada, y el costo masivo (la IA) es despreciable: eso permite un margen bruto proyectado de ~85% y un plan gratuito que funciona como canal de marketing de muy bajo costo.

El plan de crecimiento es secuencial: **Año 1** validar el embudo freemium en Chile; **Año 2** replicar el modelo en Argentina y Colombia con localización normativa; **Año 3** escalar a la América Latina hispanohablante. La meta al Año 5 es un ingreso anual recurrente (ARR) de ~US\$32 millones con 17.500 clientes de pago.

3. ¿Cómo funciona?

Justiniano es una **plataforma de tres lados** construida sobre una única infraestructura: la empresa cliente que consume el servicio, el abogado revisor que certifica los documentos y el equipo interno (administración y ventas) que opera el negocio. El flujo de valor típico es el siguiente:

- **1. Registro y onboarding.** La empresa se registra (con verificación por correo y SMS), describe su actividad y contexto, y accede al plan Gratuito.

- **2. Consulta con agentes IA.** El usuario conversa con un agente especializado (laboral, societario, tributario, etc.) que responde con fundamento en la normativa chilena y el contexto de la empresa.
- **3. Generación de documentos.** El usuario elige uno de los ~70 formatos disponibles, completa un formulario validado campo a campo y la plataforma genera el documento, descargable en Word, PDF u ODT.
- **4. Revisión por abogado (el diferenciador).** Si el usuario quiere certeza, solicita la revisión del documento por un abogado acreditado, pagando con créditos: revisión estándar en 48 horas hábiles o prioritaria en 8. El abogado trabaja en un espacio con control de cambios y anotaciones por párrafo, puede chatear con el cliente y entrega el documento firmado con una declaración.
- **5. Conversión a plan de pago.** Los límites del plan Gratuito (segundo documento del mes, agente bloqueado, historial vencido) generan momentos naturales de conversión hacia los planes de suscripción, que se cobran mediante Stripe.

Del lado del abogado, la plataforma funciona como un **marketplace de trabajo profesional**: los abogados se acreditan (verificación de identidad, título y universidad), reciben casos por asignación directa o los toman de una bolsa abierta, y cobran honorarios liquidados mensualmente contra un tarifario. Son proveedores variables, no empleados: el costo escala junto con el ingreso, una decisión de diseño operativo con impacto directo en el modelo financiero.

Del lado interno, la consola de administración muestra métricas del negocio en tiempo real y permite modificar planes, precios y límites **sin necesidad de reprogramar el software** — los límites viven en configuración, no en código — lo que convierte el pricing en una palanca de experimentación semanal. El portal del vendedor añade un CRM con cartera de clientes y proyecciones de cierre.

4. Componentes del sistema

El sistema se compone de **un backend común** (la API que concentra toda la lógica de negocio) y **tres aplicaciones web** que lo consumen, una por cada tipo de usuario. La especificación técnica v2.3 define 110 endpoints (puntos de acceso de la API) organizados en 16 módulos.

Componente	Qué hace	Usuario
Aplicación del cliente	Registro y onboarding, chat con agentes IA, generación y descarga de documentos, solicitud de revisiones, compra de planes y créditos, ajustes de cuenta.	Empresa (pyme) cliente
Portal del abogado revisor	Registro con acreditación profesional, panel de casos, bolsa de requerimientos, espacio de revisión con control de cambios y chat, honorarios y liquidaciones.	Abogado revisor externo
Consola de administración y portal del vendedor	Métricas y KPIs del negocio, gestión de usuarios y planes, panel de operación, CRM de ventas con proyecciones, gestión de la red de revisores y del tarifario.	Equipo interno (admin y ventas)
API backend (110 endpoints, 16 módulos)	Autenticación y seguridad, reglas de negocio, cuotas por plan, máquinas de estado de las revisiones, pagos, integraciones con IA y almacenamiento.	Las tres aplicaciones anteriores

Un detalle de arquitectura con valor pedagógico: los tres frontends **comparten servicios comunes** (autenticación, catálogos, verificación de salud del sistema) y el rol de cada usuario viaja dentro de su credencial (JWT). Este

principio de «mínima superficie de API» reduce la duplicación y el costo de mantenimiento — una decisión temprana de diseño que simplifica todo el proyecto posterior.

5. Tecnología utilizada

La selección tecnológica del proyecto es deliberadamente conservadora: tecnologías maduras, bien documentadas y con talento disponible en el mercado, lo que reduce el riesgo de ejecución de un equipo pequeño con plazos cortos.

Tecnología	Para qué se usa y por qué se eligió
React	Construye las tres aplicaciones web (SPA, aplicaciones de una sola página). Es la biblioteca de interfaces más extendida del mercado, lo que facilita contratar desarrolladores y compartir componentes entre los tres portales.
FastAPI (Python)	Es el marco del backend: expone la API de 110 endpoints. Genera automáticamente el contrato de la API (OpenAPI), lo que permite al equipo frontend trabajar en paralelo contra ese contrato — práctica clave del plan de desarrollo.
PostgreSQL	Base de datos relacional donde viven usuarios, documentos, revisiones, planes y pagos. Fiable, de código abierto y adecuada para reglas de negocio con transacciones (p. ej., el descuento atómico de créditos).
Microsoft Azure	La nube donde se aloja todo: servidores de la aplicación, base de datos gestionada, almacenamiento de archivos (Blob), gestión de secretos (Key Vault) y monitoreo. Contratar infraestructura como servicio evita inversión inicial en hardware.
Azure OpenAI	Provee los modelos de lenguaje que responden las consultas legales. Cada agente combina un modelo de IA con instrucciones (prompts) especializadas por área legal y la base normativa chilena. El proveedor es intercambiable por diseño, para mitigar la dependencia.
Stripe	Procesa los pagos: suscripciones mensuales en pesos chilenos, compra de packs de créditos y portal de facturación del cliente.
Azure Communication Services	Envía los correos y SMS de verificación (códigos OTP) del registro y del inicio de sesión seguro.

La seguridad es transversal al diseño: autenticación con tokens JWT y doble factor (2FA), control de acceso por roles (cliente, administrador, vendedor, abogado), límites de tasa contra abusos, validación de todos los datos de entrada y defensas específicas contra la inyección de instrucciones maliciosas en la IA (prompt injection). En un producto que maneja datos legales sensibles de empresas, la seguridad no es un añadido sino un requisito de negocio.